Andrzej Bobyk

Analiza i modelowanie danych

Zestaw ćwiczeń do samodzielnej pracy

Spis treści

1. Wprowadzenie 3
1.1. Biblioteki 3
2. Statystyki opisowe 4
2.1. Wizualizacja danych 6
3. Obsługa wartości brakujących 9
3.1. Wykrywanie braków danych 9
3.2. Uzupełnianie braków danych 10
3.3. Usuwanie danych brakujących 13
4. Znajdowanie punktów oddalonych 14
5. Wybór i inżynieria zmiennych 16
6. Kodowanie danych 18
7. Podział na zbiór uczący i testowy 22
8. Skalowanie danych 22
9. Regresja liniowa 25
10. Klasyfikacja 28
10.1. Naiwny klasyfikator Bayesa 29
10.2. Maszyny wektorów nośnych 32
10.3. Ocena jakości klasyfikacji 37
11. Ćwiczenia 40

1. Wprowadzenie

Ten skrypt wprowadzi Cię w świat modeli uczenia maszynowego oraz praktycznych metod ich budowy, trenowania i oceny za pomocą pakietu *scikit-learn*[1]. Znaczną jego część poświęcimy przygotowaniu danych do procesu trenowania – jest ono bowiem jednym z najważniejszych – o ile nie najistotniejszym – etapem każdego projektu uczenia maszynowego. Zgodnie bowiem z zasadą GIGO (wkładasz śmieci – wyjmujesz śmieci, ang. *Garbage In – Garbage Out*), nawet najbardziej wyrafinowany model nie da poprawnych wyników, jeżeli dane nie zostaną poddane właściwej i rzetelnej obróbce wstępnej (ang. *preprocessing*). Kroki związane z tym procesem będą tematem kolejnych rozdziałów niniejszego skryptu – obejmują one m.in. takie czynności, jak: eksploracja danych (w tym ich wizualizacja i wyznaczenie cech statystycznych), uzupełnianie danych brakujących, znajdowanie punktów oddalonych (ang. *outliers*), wybór i inżynieria zmiennych, kodowanie danych, podział na zbiór uczący (treningowy) i testowy, wreszcie skalowanie danych. Następnie przejdziemy do etapu budowy różnorodnych modeli uczenia maszynowego, a także ich ewaluacji, czyli oceny i porównania za pomocą tzw. metryk skuteczności modelu oraz walidacji i dostrajania. W tym kursie skupimy się na tzw. modelach predykcyjnych (realizujących zadania klasyfikacji i regresji), pozostając tym samym w obszarze uczenia nadzorowanego – zadania uczenia nienadzorowanego będą bowiem tematem następnych kursów.

Postaraj się wykonać poszczególne kroki tego procesu, opisane w niniejszym skrypcie. W jego ostatniej części znajdziesz propozycje innych przykładowych zbiorów danych, dla których ów proces należy wykonać – być może modyfikując go, stosownie do ich struktury, charakteru zmiennych, czy zawartości, a także rodzaju zadania, które tworzony model uczenia maszynowego ma zrealizować.

1.1. Biblioteki

Pewna część funkcji i metod, niezbędnych do obróbki danych, zawarta jest m.in. w znanych już nam pakietach *NumPy* oraz *pandas*, stanowiących podstawowe narzędzia z dziedziny Data Science. Pozostałe znajdziemy w pakietach, służących bezpośrednio uczeniu maszynowemu (ang. *Machine Learning* – ML), takich jak *scikit-learn*, obejmującym „klasyczne" metody ML (ale także proste sztuczne sieci neuronowe), czy *Keras*[2], zawierającym rozbudowane narzędzia do tworzenia głębokich sztucznych sieci neuronowych i ich uczenia. O ile instalacja tego ostatniego wymaga bardziej skomplikowanej procedury, o tyle pakiet *scikit-learn* instalujemy w zwykły sposób. Oczywiście, użyte w tym celu polecenie będzie zależało od tego, czy posługujemy się *PIP*-em:

```
pip install scikit-learn
```

czy *Anacondą*:

```
conda install scikit-learn
```

Pakietu *scikit-learn* nie importujemy zazwyczaj w całości, lecz – w miarę potrzeb – importując poszczególne jego składniki (funkcje lub klasy) z licznych jego podpakietów. Na przykład, chcąc przeskalować dane poprzez ich tzw. standaryzację, będziemy potrzebowali klasy `StandardScaler` z pakietu *sklearn.preprocessing*:

```
from sklearn.preprocessing import StandardScaler
```

Zwróć uwagę, że nazwa pakietu w poleceniu jego instalacji (*scikit-learn*) różni się od tej, używanej podczas jego importowania (*sklearn*). Jeżeli będziemy potrzebować więcej niż jednego elementu z tego samego podpakietu, zaimportujemy je jednym poleceniem, np.:

```
from sklearn.preprocessing import RobustScaler, LabelEncoder
```

[1] https://scikit-learn.org/stable/
[2] https://keras.io/

2. Statystyki opisowe

Zbiorem danych, który weźmiemy na tapet jako przykład do zilustrowania kolejnych kroków procesu przygotowania danych, będzie lista pasażerów słynnego statku *RMS Titanic*, który zatonął po zderzeniu z górą lodową w dniu 15 kwietnia 1912 roku[3]. Zbiór ten, dostępny w szeregu repozytoriów[4], zawiera 1309 rekordów, opisywanych przez następujące atrybuty:

- *pclass* – klasa kajuty, którą pasażer podróżował (1 – najwyższa, 3 – najniższa),
- *survived* – informacja, czy pasażer przeżył (1 – tak, 0 – nie),
- *name* – imię, nazwisko oraz tytuł pasażera,
- *sex* – płeć pasażera,
- *age* – wiek pasażera,
- *sibsp* – liczba rodzeństwa oraz małżonków, podróżujących wraz z pasażerem,
- *parch* – liczba dzieci oraz rodziców, podróżujących wraz z pasażerem,
- *ticket* – numer biletu pasażera,
- *fare* – cena biletu pasażera,
- *cabin* – numer kajuty, w której pasażer podróżował,
- *embarked* – port zaokrętowania pasażera (C – Cherbourg, Q – Queenstown, S – Southampton),
- *boat* – numer łodzi ratunkowej, w której pasażer się ewakuował,
- *body* – numer identyfikacyjny znalezionego ciała,
- *home_dest* – informacja, skąd pasażer pochodził lub dokąd zmierzał.

Pierwszą czynnością (oczywiście po zaimportowaniu niezbędnych pakietów) będzie załadowanie danych z repozytorium (w postaci pliku CSV) do ramki danych oraz wyświetlenie informacji o jej strukturze:

```
df = pd.read_csv('titanic.csv')
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1309 entries, 0 to 1308
Data columns (total 14 columns):
#   Column    Non-Null Count  Dtype
---  ------    --------------  -----
0   pclass    1309 non-null   int64
1   survived  1309 non-null   int64
2   name      1309 non-null   object
3   sex       1309 non-null   object
4   age       1046 non-null   float64
5   sibsp     1309 non-null   int64
6   parch     1309 non-null   int64
7   ticket    1309 non-null   object
8   fare      1308 non-null   float64
9   cabin     295 non-null    object
10  embarked  1307 non-null   object
11  boat      486 non-null    object
12  body      121 non-null    float64
13  home_dest 745 non-null    object
dtypes: float64(3), int64(4), object(7)
memory usage: 143.3+ KB
```

[3] https://pl.wikipedia.org/wiki/RMS_Titanic

[4] Najpełniejszą listę można znaleźć pod adresem https://www.openml.org/d/40945 – my skorzystamy z pliku lokalnego „titanic.csv".

Zwróćmy od razu uwagę na jakość danych, czyli liczbę brakujących wartości poszczególnych zmiennych. Niewielkie braki widzimy w kolumnach „fare" i „embarked", nieco więcej wartości trzeba będzie uzupełnić w kolumnie „age". Natomiast liczba wartości niepustych dla kolumn „cabin", „home_dest", „boat" oraz „body" raczej będzie skłaniała nas do eliminacji tych zmiennych – tym bardziej, że na kolejnym etapie naszym zadaniem będzie zbudowanie predykcyjnego modelu uczenia maszynowego, mającego na celu ocenę szans przeżycia pasażera Titanica – w oczywisty więc sposób użycie atrybutów „boat" i „body" spowodowałoby tzw. wyciek danych (ang. *data leakage*), czyli użycie dodatkowej informacji, która pozwoliłaby modelowi nauczyć się lub poznać coś, czego w przeciwnym razie by nie wiedział, co z kolei podważy zaufanie do szacowanej wydajności konstruowanego modelu.

Następnym krokiem będzie wyznaczenie tzw. statystyk opisowych dla wybranych kolumn danych. Możemy zrobić to globalnie, za pomocą metody `describe()`:

```
print(df[['age', 'sibsp', 'parch', 'fare']].describe())
```

```
                age          sibsp         parch          fare
count    1046.000000    1309.000000   1309.000000    1308.000000
mean       29.881135       0.498854      0.385027      33.295479
std        14.413500       1.041658      0.865560      51.758668
min         0.166700       0.000000      0.000000       0.000000
25%        21.000000       0.000000      0.000000       7.895800
50%        28.000000       0.000000      0.000000      14.454200
75%        39.000000       1.000000      0.000000      31.275000
max        80.000000       8.000000      9.000000     512.329200
```

Można też użyć w tym celu konkretnych funkcji i metod, wyliczających np. minimum – `min()`, maksimum – `max()`, średnią – `mean()`, medianę – `median()`, modę, czyli wartość najczęściej występującą – `mode()`, czy odchylenie standardowe – `std()`. Działają one zarówno dla serii, jak i całych ramek danych, licząc te cechy statystyczne kolumna po kolumnie:

```
print(df[['age', 'fare'].median())
```

```
age     28.0000
fare    14.4542
dtype:  float64
```

Metody te domyślnie pomijają przy obliczeniach wartości brakujące (`NaN`). Nieco inaczej wygląda efekt działania metody `mode()` – jako wynik otrzymujemy bowiem serię lub ramkę danych i dopiero wybierając zerowy jej wiersz dostaniemy wartości najczęściej występujące w zadanych kolumnach:

```
print(df[['pclass', 'embarked'].mode())
```

```
    pclass embarked
0        3        S
```

Nie trzeba chyba dodawać, że o ile większość tzw. funkcji agregujących daje się obliczyć tylko dla danych numerycznych, względnie – tak jak np. medianę – również porządkowych, to modę znajdziemy także dla danych o charakterze kategorycznym. Chcąc wyznaczyć kilka z tych cech jednocześnie, warto uciec się do metody `aggregate()`[5], podając jako pierwszy jej argument listę nazw metod (lub referencje do nich, a nawet słownik, jeżeli poszczególne kolumny chcielibyśmy zagregować w specyficzny dla nich sposób), które chcemy zaaplikować do kolumn danych:

[5] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.aggregate.html – analogiczna metoda istnieje dla obiektów Series: https://pandas.pydata.org/docs/reference/api/pandas.Series.aggregate.html

```
print(df[['age', 'fare'].aggregate(['mean', 'std']))
```

```
             age          fare
mean   29.881135     33.295479
std    14.413500     51.758668
```

Oprócz tych, które już wymieniliśmy, warto zwrócić uwagę na funkcję `skew()`, wyliczającą współczynnik skośności[6]rozkładu zmiennej, czy `kurtosis()`, wyznaczającą jego kurtozę[7], czyli miarę spłaszczenia. Obie te funkcje można znaleźć w bibliotece *SciPy*[8], a precyzyjniej mówiąc w podpakiecie `scipy.stats` – pozwalają one nam zorientować się, na ile rozkład statystyczny danej zmiennej podobny jest do rozkładu normalnego (zwanego też rozkładem Gaussa) – bądź na ile od niego odbiega, dając np. pewne wskazówki, co do tzw. punktów oddalonych. Oto przykład:

```
from scipy.stats import skew, kurtosis
print(skew(df['fare'], nan_policy='omit'), kurtosis(df['fare'], nan_policy='omit'))
```

```
4.362698696939623   26.9201952318479
```

Zwróćmy uwagę na użycie argumentu `nan_policy` o wartości `'omit'` – funkcje te, inaczej niż metody *pandasa*, nie pomijają bowiem domyślnie przy obliczeniach wartości `NaN`.

Istotnie różniąca się od zera i dodatnia wartość skośności dla zmiennej, oznaczającej cenę biletu każe nam wnioskować, że rozkład danych w tej kolumnie jest silnie asymetryczny, z długim „ogonem" po prawej stronie – co zresztą poparte jest faktem, że średnia wartość ceny biletu jest ponad dwukrotnie większa niż jej mediana. Rozkład ten jest ponadto smuklejszy niż rozkład normalny (leptokurtyczny), zaś intensywność wartości skrajnych jest większa niż dla rozkładu normalnego, o czym świadczy duża dodatnia wartość kurtozy – wspiera to dodatkowo naszą tezę, że dla tego atrybutu istnieje spora szansa znalezienia punktów oddalonych. *Nota bene* – spójrzmy na maksymalną cenę biletu w tabelce na stronie 5.

2.1. Wizualizacja danych

Przyjrzymy się teraz pakietom *Matplotlib* oraz jego następcy – *Seaborn* – przeznaczonym do wizualizacji danych w języku Python. Jedną z najważniejszych cech *Matplotliba* jest jego zdolność do współpracy z wieloma systemami operacyjnymi i tzw. back-endami graficznymi. *Matplotlib* obsługuje ich dziesiątki – podobnie jak i wyjściowych formatów graficznych – co oznacza, że zawsze można liczyć na jego działanie, niezależnie od używanego systemu operacyjnego lub wybranego formatu wyjściowego. To wieloplatformowe podejście typu „wszystko dla każdego" było jedną z największych zalet tego pakietu. Doprowadziło ono do powstania dużej bazy użytkowników, co z kolei zaowocowało aktywną społecznością programistów oraz potężnych narzędzi i wszechobecności w naukowym świecie Pythona. Jednakże w ostatnich latach interfejs i styl *Matplotliba* zaczęły zdradzać oznaki starzenia – stąd opracowywanie wciąż nowych pakietów, które wykorzystują potężne funkcje wewnętrzne *Matplotliba*, aby obsługiwać go za pomocą czystszych, nowocześniejszych interfejsów API. Takim przykładem może być pakiet *Seaborn*, który oferuje rozsądne wybory dotyczące stylu wykresu i domyślnych kolorów, definiuje proste funkcje wysokiego poziomu dla typowych typów wykresów statystycznych i integruje się z funkcjonalnością, udostępnianą przez pakiet *pandas*. Nawet jednak i w tym przypadku często używamy składni *Matplotliba*, aby dostosować ostateczny wygląd wykresu.

Zaczniemy, jak zwykle, od zaimportowania niezbędnych bibliotek[9]:

[6] https://pl.wikipedia.org/wiki/Wsp%C3%B3%C5%82czynnik_sko%C5%9Bno%C5%9Bci
[7] https://pl.wikipedia.org/wiki/Kurtoza
[8] https://scipy.org/
[9] Jeżeli nie zostały one zainstalowane wcześniej w używanym środowisku Pythona, należy to zrobić – analogicznie, jak opisano w podrozdziale *1.1. Biblioteki*.

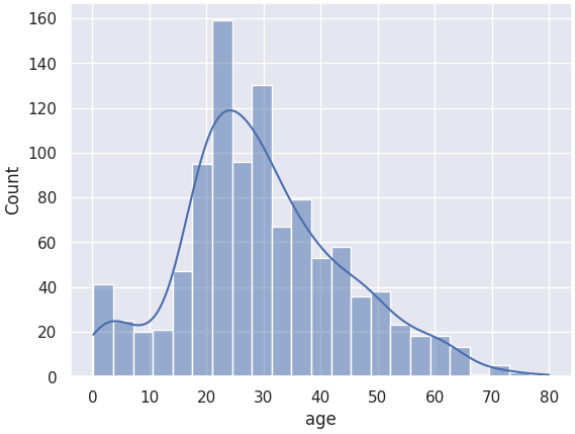
```
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

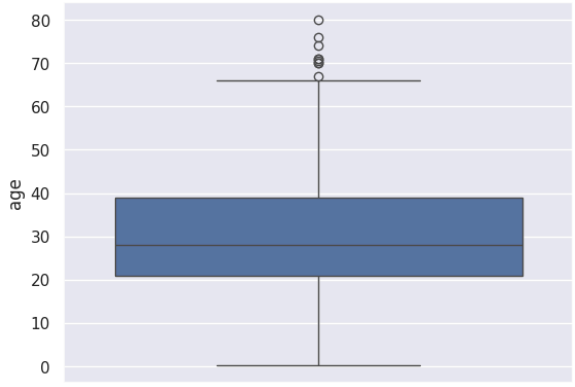
Zastosowana funkcja `sns.set()` pakietu *Seaborn* włącza zawarte w nim nowoczesne style wykresów.

Dla zobrazowania rozkładów statystycznych jednej zmiennej ciągłej najczęściej stosujemy histogramy, czasem wzbogacone o tzw. wykres gęstości (ang. *density plot*) oraz wykresy pudełkowe (ang. *box plot*)[10]. Spróbujmy stworzyć je np. dla atrybutów „age" i „fare" ramki danych `df`, zawierającej dane o pasażerach *Titanica*:

```
sns.histplot(df['age'], kde=True);
```

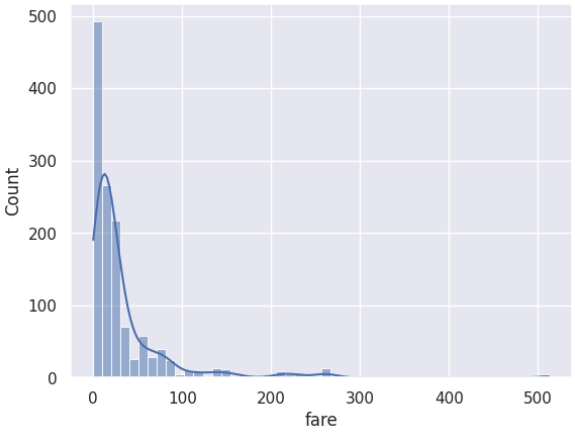
```
sns.boxplot(df['age']);
```

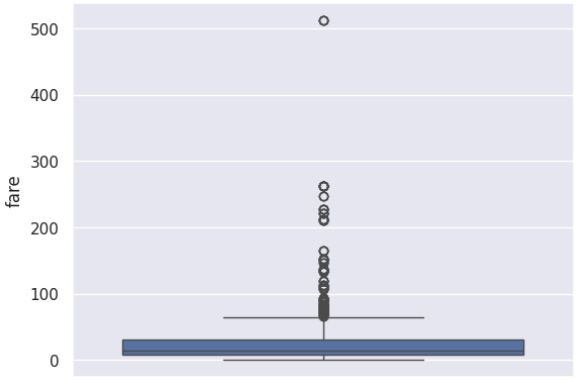





```
sns.histplot(df['fare'], kde=True, bins=50);
```

```
sns.boxplot(df['fare']);
```





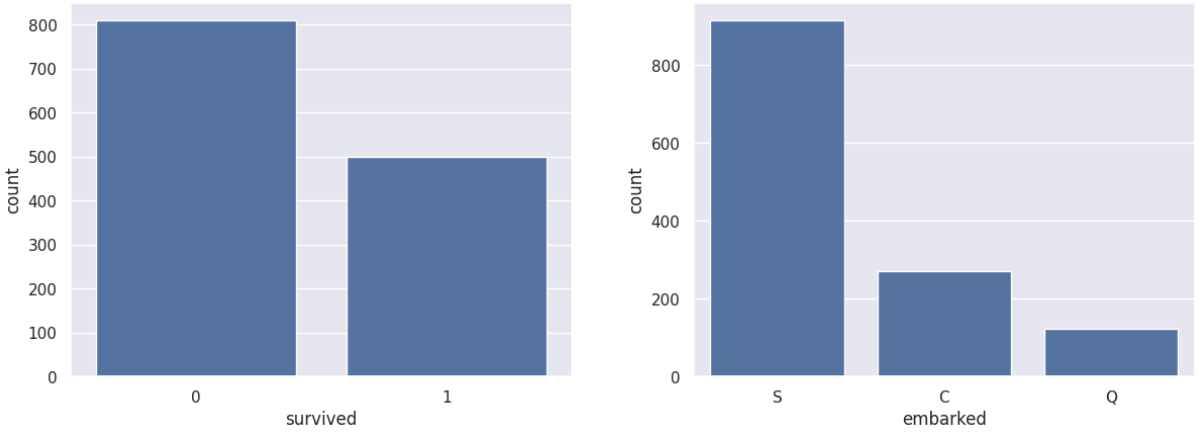
Średnik na końcu polecenia zapobiega wypisywaniu tekstu, będącego opisem obiektu, zwracanego przez funkcje lub metody, tworzące wykres. Widzimy, że te wykresy dość istotnie różnią się od siebie – jest to spowodowane istnieniem znacznie większej liczby tzw. wartości odstających (oddalonych), dla zmiennej „fare", niż dla „age". Wartości te na wykresach pudełkowych reprezentowane są przez kółeczka – samo pudełko zawarte jest pomiędzy pierwszym a trzecim kwartylem rozkładu wartości zmiennej, zaś tzw. „wąsy" reprezentują półtorakrotny zakres międzykwartylowy (patrz Rozdział 4).

[10] https://miroslawmamczur.pl/category/wykresy/

W przypadku zmiennych dyskretnych, takich, jak np. „survival”, czy „embarked”, do ich wizualizacji zastosujemy wykresy słupkowe (ang. *bar plot*, *count plot*). Przypominają one nieco histogramy – jednak w przypadku tych ostatnich możemy „sterować” liczbą słupków (a tym samym ich szerokością) za pomocą argumentu `bins`, natomiast dla wykresów słupkowych, liczba słupków jest wprost dana liczbą różnych wartości zmiennej, czyli kategorii (klas).

```
sns.countplot(df, x='survived');
```

```
sns.countplot(df, x='embarked');
```



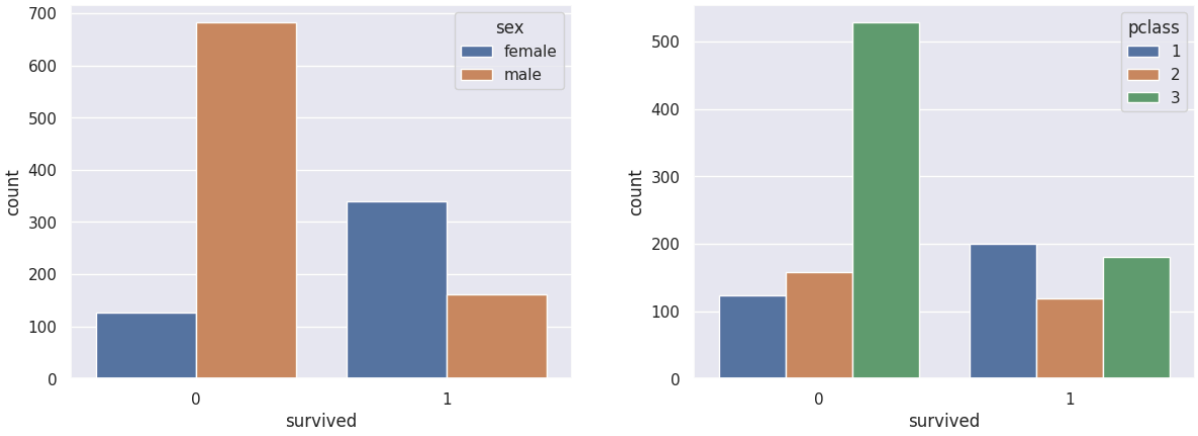
Zwróć uwagę na sposób użycia tej funkcji, uwzględniający strukturę ramki danych, dla której wykonywany jest wykres.

Pierwszy rysunek niesie ze sobą informację, bardzo istotną z punktu widzenia uczenia maszynowego. Chodzi tu mianowicie o tzw. zrównoważenie klas, czyli proporcje liczby przypadków uczących, przypadających na różne wartości etykiety (zmiennej celu). W przypadku modeli predykcyjnych najlepiej jest, gdy te proporcje są w miarę równe – w innym przypadku model najczęściej działa źle i z dużą ostrożnością należy podchodzić do metryk oceny jakości takiego modelu. Można temu zapobiec poprzez tzw. augmentację danych (uzupełnianie) – będzie to tematem bardziej zaawansowanych kursów.

Wykresy, takie jak powyższe, możemy wzbogacić o dodatkowe informacje, co pozwoli nam dokonać wstępnej analizy danych, nawet bez konieczności budowy modeli uczenia maszynowego. Przyjrzyjmy się, dla przykładu, jak płeć pasażera, czy klasa kajuty, miały wpływ na szanse jego przeżycia:

```
sns.countplot(df, x='survived', hue='sex');
```

```
sns.countplot(df, x='survived', hue='pclass', palette='tab10');
```



3. Obsługa wartości brakujących

Różnica między danymi znalezionymi w wielu samouczkach a danymi ze świata rzeczywistego polega na tym, że te drugie rzadko są czyste i jednorodne. W szczególności, w wielu interesujących zbiorach danych, będzie brakować pewnej ilości danych. Sprawę jeszcze bardziej komplikuje fakt, że różne źródła danych mogą w różny sposób wskazywać braki danych. W tym rozdziale dokonamy pewnych ogólnych rozważań, dotyczących brakujących danych i zademonstrujemy pewne narzędzia do ich obsługi.

Twórcy pakietu *pandas* zdecydowali się takie dane reprezentować za pomocą specjalnych wartości, zwanych strażnikami (ang. *sentinel*), najczęściej przyjmującymi wartości `None` lub `NaN` (a w przypadku danych o charakterze czasowym, także `NaT`). O ile wartość `None` jest tzw. singletonowym obiektem *Pythona*, traktowanym w kodzie jako pusta referencja, o tyle nie można jej użyć w dowolnej tablicy *NumPy*, czy serii lub ramce danych *pandasa*, a jedynie w tablicach z typem danych *object* (tj. tablicach obiektów *Pythona*). Oznacza to jednak również, że jeśli wykonamy agregację, taką jak `sum()` lub `min()` w tablicy z wartością `None`, zazwyczaj pojawi się błąd niezgodności typów. Znacznie wygodniej więc będzie korzystać z innej reprezentacji brakujących danych, `NaN` (ang. *Not-a-Number*), będącej specjalną wartością zmiennoprzecinkową w standardzie IEEE, rozpoznawaną m.in. przez (stanowiący podstawę *pandasa*) pakiet *NumPy*, gdzie jest ona obecna w typie `float64` jako `np.nan`. Należy pamiętać, że nie ma równoważnej wartości `NaN` dla liczb całkowitych, ciągów znaków ani innych typów – wynika to właśnie stąd, że twórcy pakietu *NumPy* wybierając standard IEEE, mieli na celu szybką obsługę operacje arytmetycznych za pomocą kodu skompilowanego. Trzeba mieć też świadomość, że `NaN` jest trochę jak wirus – infekuje każdy inny obiekt, którego dotknie. Niezależnie od operacji, wynikiem arytmetyki z `NaN` będzie kolejna `NaN`.

Pakiet *pandas* dobrze radzi sobie z oboma sposobami reprezentowania wartości brakujących (zarówno `None`, jak i `np.nan`), dokonując – jeśli potrzeba – konwersji między nimi i traktując je jako zasadniczo wymienne w celu wskazania brakujących wartości. Dla typów, które nie mają dostępnej wartości strażnika, *pandas* automatycznie rzutuje typ w górę, gdy obecne są wartości puste. Na przykład, jeśli ustawimy wartość w serii liczb całkowitych na `np.nan`, seria ta zostanie w całości automatycznie „wypromowana" do typu `float64`, jednocześnie wartości None zostaną przekształcone na `np.nan`. Także metody agregujące są „świadome" istnienia wartości pustych i domyślnie pomijają je przy wyliczaniu odpowiedniej wartości w kolumnie (zachowanie to można zmienić, nadając argumentowi `skipna` wartość `False`).

Istnieje kilka przydatnych metod wykrywania, usuwania i zastępowania wartości pustych w strukturach danych *pandasa* – oto one:

- `isnull()`: generuje maskę boolowską, wskazującą brakujące wartości;
- `notnull()`: przeciwieństwo `isnull()`;
- `dropna()`: zwraca przefiltrowaną wersję danych, usuwając wiersze (lub też kolumny, dla obiektów `DataFrame`), zawierające brakujące dane;
- `fillna()`: zwraca kopię serii lub ramki danych z uzupełnionymi brakującymi wartościami.

W kolejnych podrozdziałach zajmiemy się opisem i wyjaśnieniem działania tych metod na przykładach.

3.1. Wykrywanie braków danych

Weźmy jako przykład następującą serię danych (ze względu na mieszanie danych różnych typów, będzie ona reprezentowana jako tablica typu `object`):

```
data = pd.Series([1, np.nan, 'hello', None])
print(data.isnull())
```

```
0       False
1        True
2       False
3        True
dtype:  bool
```

Jak już wiemy z poprzedniego skryptu, masek logicznych możemy używać bezpośrednio do indeksowanie serii lub ramki danych:

```
print(data[data.notnull()])
```

```
0       1
2       hello
dtype:  object
```

Metody `isnull()` i `notnull()` dają podobne wyniki (maski logiczne w postaci dwuwymiarowych tablic) dla obiektów `DataFrame`.

3.2. Uzupełnianie braków danych

W przypadku, gdy w kolumnie danych znajdują się wartości brakujące, możemy próbować je uzupełnić (czynność tę nazywa się często **imputacją**) – oczywiście, w sytuacji, gdy jest ich stosunkowo niewiele, a dana zmienna może mieć znaczenie w procesie budowania modelu. Wartość, którą uzupełniamy może być pojedynczą liczbą, taką jak zero, lub może to być pewnego rodzaju uśrednienie lub interpolacja dobrych wartości. Można to zrobić, używając metody `isnull()` do wygenerowania maski, ale ponieważ jest to powszechna operacja, *pandas* udostępnia metodę `fillna()`, która zwraca kopię tablicy z zastąpionymi wartościami pustymi.

Rozważmy następującą serię danych:

```
data = pd.Series([1, np.nan, 2, None, 3], index=list('abcde'))
print(data)
```

```
a   1.0
b   NaN
c   2.0
d   NaN
e   3.0
dtype: float64
```

Wartości puste możemy wypełnić pojedynczą wartością, np. zerem (chociaż częściej użyjemy np. mediany, wartości średniej, czy mody – sposób uzupełniania będzie analogiczny):

```
print(data.fillna(0))
```

```
a   1.0
b   0.0
c   2.0
d   0.0
e   3.0
dtype: float64
```

Czasem, szczególnie w przypadku danych o charakterze sekwencyjnym (np. dla szeregów czasowych) brakujące wartości są blisko związane z wartościami sąsiednimi i niewiele się od nich różnią. Możemy

w tej sytuacji zażądać tzw. wypełniania w przód, aby propagować ostatnią znaną wartość do kolejnych pustych elementów serii (z oczywistego powodu nie pozwoli nam to wypełnić brakujących wartości w początkowych komórkach serii):

```
data.fillna(method='ffill')
```

```
a   1.0
b   1.0
c   2.0
d   2.0
e   3.0
dtype: float64
```

Do metody `fillna()` możemy przekazać też dodatkowe argumenty, np. specyfikujące, ile maksymalnie kolejnych pustych komórek zostanie wypełnionych (`limit` - domyślnie brak), czy określający, że zastępowanie danych ma się odbyć w obszarze pamięci, gdzie obiekt `Series` lub `DataFrame` się znajduje, bez tworzenia jego kopii. Podobnie postąpimy przy wypełnianiu wstecz (analogicznie, zostaną pominięte brakujące wartości w końcowych komórkach serii):

```
data.fillna(method='bfill')
```

```
a   1.0
b   2.0
c   2.0
d   3.0
e   3.0
dtype: float64
```

W przypadku ramek danych mamy podobne opcje, ale możemy też określić wymiar – wierszy, lub kolumn (tzw. oś, ang. *axis*) – wzdłuż którego odbywają się wypełnienia: argument `axis=0`, `axis='index'` lub `axis='rows'` (wartość domyślna) oznacza wypełnianie **wierszami w dół**, zaś `axis=1` lub `axis='columns'`– wypełnianie **kolumnami w prawo** (jest to konwencja powszechnie obowiązująca, zarówno dla dwuwymiarowych tablic *NumPy*, jak i ramek danych *pandasa*):

```
df = pd.DataFrame([[1,      np.nan,  2,  np.nan],
                   [2,      3,       5,  np.nan],
                   [np.nan, 4,       6,  np.nan]])
print(df.fillna(method='ffill', axis=1))
```

```
     0    1    2    3
0  1.0  1.0  2.0  2.0
1  2.0  3.0  5.0  5.0
2  NaN  4.0  6.0  6.0
```

Jak już zaznaczyliśmy, jeśli poprzednia wartość nie jest dostępna podczas wypełniania do przodu, wartość pusta pozostaje niezmieniona.

Trochę bardziej zaawansowany sposób uzupełniania zademonstrujemy teraz na przykładzie danych pasażerów *Titanica*. Chcąc wypełnić wartości brakujące w kolumnie „age", możemy wprost użyć mediany tej zmiennej (jest to wybór zazwyczaj lepszy, niż użycie średniej, zwłaszcza dla rozkładów asymetrycznych). W celu porównania różnych podejść do imputacji danych, nie będziemy modyfikować wartości bezpośrednio w kolumnie „age", lecz stwórzmy nową kolumnę i zobaczmy, co zostało wstawione w miejsce braków:

```
df['age_1'] = df['age'].fillna(df['age'].median())
print(df['age_1'][df['age'].isnull()]
```

```
15    28.0
37    28.0
40    28.0
46    28.0
59    28.0
...    ...
1293  28.0
1297  28.0
1302  28.0
1303  28.0
1305  28.0
Name: age_1, Length: 263, dtype: float64
```

Jak widzimy, w miejsce brakujących danych pojawiła się stała wartość 28.0. Można jednak przypuszczać, że mediana wieku dla kobiet będzie inna, niż dla mężczyzn – wykorzystajmy tę informację w procesie uzupełniania brakujących danych w kolumnie „age". W tym celu pogrupujemy[11] wiersze danych, w zależności od płci pasażera i wyliczymy medianę niezależnie da każdej z grup:

```
print(df.groupby('sex')['age'].median())
```

```
sex
female  27.0
male    28.0
Name: age, dtype: float64
```

Uzupełnijmy teraz (tworząc nową kolumnę „age_2") brakujący wiek odpowiednią wartością, w zależności od płci pasażera. Użyjemy w tym celu metody transform(), która – w naszym wypadku – zadziała w ten sposób, że do każdej wartości serii zastosuje daną funkcję, jednak wyliczaną dla każdej z grup niezależnie[12]:

```
df['age_2'] = df['age'].fillna(df.groupby('sex')['age'].transform('median'))
print(df[['sex', 'age_2']][df['age'].isnull()])
```

```
         sex    age_2
15      male     28.0
37      male     28.0
40      male     28.0
46      male     28.0
59    female     28.0
...      ...      ...
1293    male     28.0
1297    male     28.0
1302    male     28.0
1303    male     28.0
1305  female     28.0
```

[11] https://pandas.pydata.org/docs/reference/api/pandas.Series.groupby.html

[12] https://pandas.pydata.org/docs/reference/api/pandas.core.groupbySeriesGroupBy.transform.html – obiekt, na którym ta metoda działa (`SeriesGroupBy`), powstaje w wyniku zastosowania metody `groupby()` na obiekcie `Series`.

Jak widać powyżej, teraz wstawiana w miejsce braków danych w kolumnie „age” wartość mediany jest uzależniona od płci pasażera.

3.3. Usuwanie danych brakujących

Jeżeli braków w danych jest na tyle dużo, że ich uzupełnianie traci sens (zazwyczaj mówimy tu o progu na poziomie 20–30%), należy rozważyć usunięcie wierszy i/lub kolumn o niskiej jakości danych. Służy temu metoda `dropna()`[13] – domyślnie usuwa ona wszystkie wiersze, które zawierają wartości puste. Jej działanie zademonstrujemy na następującym przykładzie:

```
df = pd.DataFrame([[1,      np.nan, 2],
                   [2,      3,      5],
                   [np.nan, 4,      6]])
print(df.dropna())
```

```
    0    1  2
1   2.0  3.0  5
```

Alternatywnie, kładąc argument `axis='columns'`, możemy pozbyć się wszystkich (a używając argumentu `subset` – wybranych) kolumn, zawierających przynajmniej jedną wartość pustą:

```
print(df.dropna(axis='columns'))
```

```
    2
0   2
1   5
2   6
```

Niestety, powoduje to również utratę dobrych danych – chcielibyśmy raczej usunąć wiersze lub kolumny ze wszystkimi wartościami pustymi lub zawierającymi je w większości. Sposób postępowania można określić za pomocą argumentów `how` lub `thresh`, które umożliwiają precyzyjną kontrolę liczby „przepuszczanych” wartości nieznanych. Wartość domyślna pierwszego z nich to `'any'`, co oznacza usunięcie dowolnego wiersza lub kolumny (w zależności od wartości argumentu `axis`) zawierającego przynajmniej jedną wartość pustą. Możemy także wyspecyfikować `how='all'`, co spowoduje usunięcie tylko tych wierszy/kolumn, które zawierają wyłącznie wartości puste. Weźmy ramkę danych, zdefiniowaną na stronie ??? i zobaczmy, jaki efekt mają wyżej wymienione argumenty:

```
print(df.dropna(axis='columns', how='all'))
```

```
     0    1  2
0   1.0  NaN  2
1   2.0  3.0  5
2   NaN  4.0  6
```

Aby uzyskać bardziej szczegółową kontrolę nad tym, które wiersze/kolumny zostaną usunięte, użyjemy teraz argumentu `thresh`, pozwalającego określić minimalną liczbę obecnych wartości niepustych w wierszu/kolumnie, wymaganych do ich pozostawienia:

```
print(df.dropna(axis='rows', thresh=3))
```

```
     0    1  2    3
1  2.0  3.0  5  NaN
```

[13] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html

4. Znajdowanie punktów oddalonych

Znalezienie, a następnie odfiltrowanie lub przycięcie wartości odstających jest niezwykle istotne z punktu widzenia zdolności modelu uczenia maszynowego do generalizacji. Zadanie to polega w dużej mierze na stosowaniu odpowiednich operacji na kolumnach naszego zbioru danych. Rozważmy ramkę danych, zawierającą po 1000 wartości dla czterech zmiennych, zawierających losowe dane o rozkładzie normalnym:

```
data = pd.DataFrame(np.random.standard_normal((1000, 4)))
print(data.describe())
```

```
                0            1            2            3
count 1000.000000 1000.000000 1000.000000 1000.000000
mean     0.023861    -0.049586     0.010283    -0.026573
std      0.994878     1.001341     0.964438     1.011160
min     -4.074555    -2.765882    -2.967156    -3.140329
25%     -0.650550    -0.762624    -0.621684    -0.720088
50%      0.026265    -0.028179     0.039011    -0.035552
75%      0.710735     0.596092     0.651407     0.660637
max      2.663821     3.369567     2.962314     2.623787
```

Zazwyczaj dla rozkładów danych, zbliżonych do normalnego, wartości odstające definiujemy jako te, które są odległe o więcej niż trzykrotność odchylenia standardowego (tzw. reguła 3σ) od wartości średniej (dla rozkładu normalnego, statystycznie 99.73% obserwacji powinno mieścić się w tym przedziale). Można je więc znaleźć za pomocą następującego polecenia:

```
mask = ((data - data.mean()).abs() > 3.0 * data.std()).any(axis='columns')
print(data[mask])
```

```
            0         1         2         3
128 -4.074555  0.964033  0.951065 -0.866215
169  0.770473 -0.416531 -2.967156  0.182272
207 -3.252989 -2.164181 -0.526906  1.861919
235  1.057038  2.957640 -0.357571 -0.943444
264 -1.218311 -0.403980  0.120439 -3.096313
443  0.209095 -0.948201  2.962314  0.353776
457 -0.714426  3.369567  0.037645  0.951872
579  1.883219  0.181085 -0.497338 -3.140329
991 -3.089797 -1.381320  0.578250  0.015426
```

Użyta tu metoda `any()`[14] zwraca wartość `True`, jeżeli przynajmniej jeden element w wierszu (argument `axis= 'columns'`, czyli „w poprzek kolumn"), spełnia zadany warunek. Możemy teraz użyć tak zdefiniowanej maski, by przyciąć wartości odstające do wcześniej określonego przedziału 3σ (dla prostoty załóżmy, że średnia wartość dla każdej z kolumn wynosi 0, zaś odchylenie standardowe 3) – zwróćmy uwagę, jakie wartości minimalne i maksymalne pojawią się dla tak przekształconych kolumn (funkcja `np.sign()` zwraca wartość 1 lub –1 w zależności od tego, czy jej argument jest dodatni, czy ujemny):

```
data[mask] = np.sign(data) * 3.0
print(data.describe())
```

[14] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.any.html

```
                0            1            2            3
count    1000.000000  1000.000000  1000.000000  1000.000000
mean        0.029291    -0.054744     0.012983    -0.018892
std         1.013166     1.026794     0.995817     1.038378
min        -3.000000    -3.000000    -3.000000    -3.000000
25%        -0.650550    -0.770640    -0.625214    -0.720088
50%         0.026265    -0.028179     0.040519    -0.035552
75%         0.714604     0.597166     0.664812     0.666875
max         3.000000     3.000000     3.000000     3.000000
```

Równie często stosowaną, co reguła 3σ, definicją wartości odstających, jest tzw. reguła IQR. Akronim ten oznacza tzw. zakres międzykwartylowy (ang. *Inter-Quartile Range*) i jest definiowany jako różnica pomiędzy trzecim (Q3) a pierwszym (Q1) kwartylem (patrz wartości dla wierszy oznaczonych odpowiednio „75%" i „25%" w powyższej tabeli) rozkładu wartości zmiennej. Przyjmuje się, że obserwacje, leżące niżej (wyżej) niż 1.5xIQR od wartości pierwszego (trzeciego) kwartyla, traktuje się jako punkty oddalone (odpowiada to „regule 2.7σ" dla rozkładu normalnego, dając statystycznie 99.7% wartości, mieszczących się w tym przedziale). Zdefiniujmy funkcję, wyliczającą IQR dla serii danych – posłużymy się tu metodą `quantile()`[15], wyznaczającą tzw. kwantyle[16] rzędu q (jest to parametr metody). W naszym wypadku będziemy potrzebowali kwartyli, czyli – inaczej mówiąc – kwantyli rzędu $\frac{1}{4}$ i $\frac{3}{4}$.

```
def iqr(s):
    return s.quantile(0.75) - s.quantile(0.25)
```

Możemy teraz użyć tej funkcji, analogicznie jak `np.std()` do znalezienia punków oddalonych – tym razem na tapet weźmy kolumnę „fare" z ramki danych, opisujących pasażerów *Titanica*:

```
print(iqr(df['fare']))
```

```
23.379199999999997
```

Znajdźmy teraz górną granicę przedziału wartości opłat za bilet, poza którą będą się mieściły punkty oddalone:

```
limit = df['fare'].quantile(0.75) + 1.5 * iqr(df['fare'])
print(limit)
```

```
66.34379999999999
```

Zobaczmy, którzy pasażerowie zapłacili wygórowaną cenę za swoje bilety (dla przejrzystości wypiszmy tylko fragment ramki danych):

```
mask = df['fare'] > limit
print(df.loc[mask, ['name', 'pclass', 'fare']])
```

```
                                          name  pclass      fare
0                      Allen, Miss. Elisabeth Walton       1  211.3375
1                     Allison, Master. Hudson Trevor       1  151.5500
2                       Allison, Miss. Helen Loraine       1  151.5500
3               Allison, Mr. Hudson Joshua Creighton       1  151.5500
4    Allison, Mrs. Hudson J C (Bessie Waldo Daniels)       1  151.5500
...                                              ...     ...       ...
```

[15] https://pandas.pydata.org/docs/reference/api/pandas.Series.quantile.html
[16] https://pl.wikipedia.org/wiki/Kwantyl

Przycięcie ceny biletu do górnej granicy zakresu cen „normalnych” nie będzie teraz zadaniem trudnym (dolna granica jest ujemna, a ponieważ cena biletu zawsze jest wartością dodatnią, nie ma obserwacji poniżej Q1 − 1.5 × IQR):

```
df.loc[mask, 'fare'] = limit
print(df['fare'].describe())
```

```
count    1308.000000
mean       24.287208
std        20.795666
min         0.000000
25%         7.895800
50%        14.454200
75%        31.275000
max        66.343800
Name: fare, dtype: float64
```

Przycinanie wartości odstających można też prosto zrealizować za pomocą metody `clip()`[17] – w poniższym (alternatywnym do wcześniejszego i dającym identyczny wynik) przykładzie operacja przycięcia zostanie wykonana *in situ*, tj. bez tworzenia kopii danych modyfikowanej zmiennej:

```
df['fare'].clip(upper=limit, inplace=True)
```

5. Wybór i inżynieria zmiennych

Wybierając zmienne, które następnie będziemy używać w procesie uczenia, należy zwrócić uwagę nie tylko na aspekt jakości danych, czy wycieku informacji (patrz strona 5) ale również na to, czy nie niosą one ze sobą informacji podobnych, a wręcz identycznych – co również mogłoby negatywnie wpłynąć na jakość modelu. Zacznijmy jednak od wydzielenia tak zwanej zmiennej celu (określanej też mianem etykiety lub zmiennej zależnej), której wartości będziemy w przyszłości przewidywać, na bazie zbudowanego i wytrenowanego predykcyjnego modelu uczenia maszynowego[18]. W przypadku danych *Titanica* będzie to kolumna „survival”, zawierająca informację o tym, czy pasażerowi udało się przeżyć katastrofę (tradycyjnie wektor zmiennej celu oznaczamy symbolem y) oraz usunięcia tej kolumny z ramki danych (wraz z kolumnami o niskiej jakości danych oraz tymi, które powodowałyby wyciek danych[19]) – będzie ona po tej operacji zawierała tylko tzw. zmienne uczące:

```
y = df['survived'].values
df.drop(columns=['survived', 'cabin', 'home_dest', 'boat', 'body'],
        inplace=True)
print(y.shape, df.shape)
```

```
(1309,) (1309, 9)
```

Jak widać, pozostało nam 9 zmiennych uczących, które możemy teraz poddać dalszej obróbce. Sprawdźmy, czy któreś z nich nie są związane ze sobą – w tym celu wyliczmy tzw. współczynnik korelacji liniowej r Pearsona[20]. Możemy to zrobić dla wybranej pary zmiennych (numerycznych), stosując

[17] https://pandas.pydata.org/docs/reference/api/pandas.Series.clip.html#pandas.Series.clip – istnieje również analogiczna metoda, pozwalająca przyciąć wartości odstające dla wszystkich kolumn ramki danych: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.clip.html

[18] Pozostajemy więc w sferze tzw. uczenia nadzorowanego (z nauczycielem), w odróżnieniu od uczenia nienadzorowanego, gdzie zmienna celu (etykieta) nie jest dana.

[19] Kolumna „ticket”, zawierająca praktycznie wartości losowe, także nie będzie przydatna jako zmienna ucząca.

[20] https://pl.wikipedia.org/wiki/Wsp%C3%B3%C5%82czynnik_korelacji_Pearsona

metodę `corr()` z klasy `Series`[21], bądź od razu wyznaczyć całą macierz korelacji (dla każdej z par), za pomocą analogicznej metody z klasy `DataFrame`[22]:

```
print(df.corr(numeric_only=True))
```

```
            pclass       age      sibsp      parch       fare
pclass    1.000000 -0.408106   0.060832   0.018322 -0.726035
age      -0.408106  1.000000 -0.243699 -0.150917   0.220824
sibsp     0.060832 -0.243699   1.000000   0.373587   0.321285
parch     0.018322 -0.150917   0.373587   1.000000   0.299143
fare     -0.726035  0.220824   0.321285   0.299143   1.000000
```

Zależności między zmiennymi, dane poprzez ich wzajemne korelacje, możemy w przystępny sposób zwizualizować – użyjemy w tym celu funkcji `heatmap()`[23] z pakietu *Seaborn*:

```
sns.heatmap(df.corr(numeric_only=True), cmap='RdBu_r', annot=True, vmin=-1, vmax=1);
```



Możemy zauważyć dość silną (ujemną) korelację pomiędzy klasą kabiny a ceną biletu, co nie powinno nas dziwić – im bowiem wyższa klasa kabiny (mniejsza wartość zmiennej „pclass"), tym droższy bilet. Należy więc zastanowić się, czy jednej z tych zmiennych nie usunąć – a przynajmniej zbadać, jaki będzie to miało wpływ na jakość budowanego modelu ML. Interpretację pozostałych wartości w macierzy korelacji pozostawiamy Czytelnikowi.

W procesie inżynierii zmiennych tworzymy nowe atrybuty na bazie już istniejących. Możemy na przykład użyć informacji z kolumny „name" (spójrz na stronę 13 i przeanalizuj strukturę danych, w niej zapisanych) w celu wydobycia tytułu pasażera. Utwórzmy kolumnę „title", korzystając z następującego polecenia (i wypisując następnie listę napotkanych tytułów):

[21] https://pandas.pydata.org/docs/reference/api/pandas.Series.corr.html
[22] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.corr.html
[23] https://seaborn.pydata.org/generated/seaborn.heatmap.html

```
df['title'] = df['name'].str.split('[,.]').str[1].str.strip()
print(df['title'].unique())
```

```
['Miss' 'Master' 'Mr' 'Mrs' 'Col' 'Mme' 'Dr' 'Major' 'Capt' 'Lady'
 'Sir' 'Mlle' 'Dona' 'Jonkheer' 'the Countess' 'Don' 'Rev' 'Ms']
```

Zastosowana tu metoda `str.split()` dzieli każdy element (łańcuch znaków) w kolumnie „name" na tzw. tokeny, jako separator przyjmując przecinek lub kropkę (dane jako tzw. wyrażenie regularne[24] poprzez argument tej metody). Następnie za pomocą wyrażenia `str[1]` wybierany jest odpowiedni token z listy i wreszcie pozbawiany wiodących oraz końcowych spacji metodą `str.strip()`. Otrzymaliśmy w ten sposób dodatkową informację, która może być istotna dla przewidzenia szans przeżycia pasażera *Titanica*.

Innym przykładem inżynierii zmiennych byłoby utworzenie atrybutu „mpc", mającego na celu uwypuklenie szans na przeżycie dzieci i osób z pierwszej klasy. Będzie on wynikiem mnożenia wieku danej osoby i klasy, w której podróżowała. Można przypuszczać, że 5-letnie dziecko podróżujące w pierwszej klasie (*mpc* = 5), miało dużo większe szanse na przetrwanie katastrofy niż 70-letni mężczyzna podróżujący klasą trzecią (*mpc* = 210). Możemy też, dajmy na to, stworzyć zmienną „family", określającą wielkość rodziny, podróżującej wspólnie, poprzez dodanie do siebie dwóch istniejących atrybutów: „parch" i „sibsp", oraz liczby 1, odpowiadającej za daną osobę. Intuicyjnie można przyjąć, że wielkość rodziny mogła mieć znaczący wpływ na to, czy komuś udało się przeżyć, czy też nie:

```
df['mpc'] = df['pclass'] * df['age']
df['family'] = df['parch'] + df['sibsp'] + 1
```

6. Kodowanie danych

Zanim przejdziemy stricte do kodowania danych, przypomnijmy sobie, jakie ich rodzaje wyróżniamy. Mamy bowiem zmienne o charakterze:

- jakościowym (nazywane również zmiennymi wyliczeniowymi, czynnikowymi lub kategorycznymi), przyjmujące określoną liczbę wartości (najczęściej nieliczbowych) z pewnego zdefiniowanego zbioru (dziedziny) – można je dalej podzielić na:
  - dychotomiczne (dwumianowe, binarne) np. płeć (kobieta/mężczyzna),
  - nominalne (jakościowe nieuporządkowane) np. marka samochodu, kolor oczu,
  - porządkowe, np. poziom wykształcenia (podstawowe/średnie/wyższe/podyplomowe);
- ilościowym (inaczej numerycznym – o wartościach rzeczywistych lub całkowitych), wśród których można dodatkowo wyróżnić:
  - zliczenia (liczba wystąpień pewnego zjawiska, opisywana liczbą całkowitą), np. liczba lat nauki, liczba wypadków,
  - zmienne ilorazowe, czyli zmienne mierzone w skali, w której można dzielić wartości (ilorazy mają sens) – np. długość w metrach (coś jest 2 razy dłuższe, 10 razy krótsze itp.),
  - zmienne przedziałowe (nazywane też interwałowymi), mierzone w skali, w której można odejmować wartości (wyznaczać długość przedziału) – np. daty (można liczyć czas pomiędzy dwoma datami ale nie ma sensu ich dzielić), czy temperatura (można liczyć różnice temperatur w stopniach Celsjusza, ale nie ma sensu liczyć ilorazu[25]).

W zależności od tego co opisuje zmienna, można na niej wykonywać inne operacje. Dla zmiennych w skali uporządkowanej można wyznaczać wartość minimalną, maksymalną, środkową – ale może nie mieć sensu liczenie średniej. Warto też dodać, że różnice zmiennych na skali przedziałowej (skali z jednostką), a więc długości przedziałów, są już na skali ilorazowej (skali z bezwzględną wartością 0).

[24] https://pl.wikipedia.org/wiki/Wyra%C5%BCenie_regularne

[25] Chyba, że temperaturę wyrazimy w tzw. skali bezwzględnej i podamy w kelwinach.

Przykładowo, dat 2016 r. n.e. i 1995 r. n.e. nie można dzielić (jaki sens miałby taki iloraz?) ale można odejmować. Różnica to 11 lat a lata można już dzielić (11 lat to dwa razy mniej niż 22 lata).

Od charakteru zmiennej będzie też zależał sposób jej kodowania w procesie przygotowania danych do uczenia maszynowego. Zacznijmy od danych numerycznych – często jest tak, że nadmiar informacji nie pomaga, a wręcz przeszkadza w uporządkowaniu wiedzy[26]. Nie inaczej jest z metodami uczenia maszynowego – zbyt duża liczba zmiennych[27], a nawet zbyt duża liczba ich wartości może spowodować, że uczenie modelu przebiega wolno i jest on podatny na tzw. przeuczenie (wspominaliśmy już o tym przy okazji badania korelacji zmiennych uczących – patrz rozdział poprzedni). Jednym z podejść, które pozwala zapobiec tym niepożądanym efektom jest przekształcenie zmiennych numerycznych w kategoryczne za pomocą tzw. binningu. Słowo to pochodzi z języka angielskiego i dosłownie oznacza „podział na kosze", co w dowolnym tłumaczeniu można by rozumieć jako „segregacja danych". Weźmy dla przykładu kolumnę „age_2" ze zbioru danych *Titanica* – nie jest w gruncie rzeczy istotne dla szans przeżycia pasażera, czy miał on dokładnie 23, czy 24 lata, ale to, czy był dzieckiem, młodzieńcem (młódką?), osobą dorosłą, czy staruszkiem/staruszką. Skorzystamy tu z funkcji `cut()`[28] pakietu *pandas*, wprowadzając wyżej wymienione kategorie wiekowe – oczywiście, granice przedziałów wybraliśmy arbitralnie, by pokazać sam proces binningu:

```
df['age_cat'] = pd.cut(df['age_2'], bins=[0, 13, 18, 60, 100],
    labels=['dziecko', 'nastolatek', 'dorosły', 'staruszek'])
print(df['age_cat'].unique())
```

```
['dorosły', 'dziecko', 'staruszek', 'nastolatek']
  Categories (4, object): ['dziecko' < 'nastolatek' < 'dorosły' < 'staruszek']
```

Otrzymane kategorie są uszeregowane, więc nowa zmienna „age_cat" ma w istocie charakter porządkowy. Zwróć uwagę, że etykiet przedziałów powinno być o jedną mniej niż wartości ich granic. Domyślnie metoda `cut()` dolne granice przedziałów przyjmuje inkluzywnie, zaś górne – ekskluzywnie, chyba, że zmienimy domyślną wartość argumentu right z `True` na `False`. Podobne znaczenie ma argument `include_lowest` o domyślnej wartości `False`, za pomocą którego określamy, czy dolna granica pierwszego przedziału ma być traktowana wyłącznie, czy włącznie.

Przyjrzyjmy się teraz danym kategorycznym nieuporządkowanym – w najprostszym przypadku dychotomicznym, które można wprost zakodować binarnie, za pomocą zer i jedynek. Pytanie – po co to robimy? Otóż, wiele algorytmów i metod uczenia maszynowego (w szczególności sztuczne sieci neuronowe) wymaga danych numerycznych, zatem nie będzie w stanie działać, jeżeli w kolumnie danych uczących pojawią się ciągi znaków „kobieta" i „mężczyzna" (lub „female" i „male" – jak dla zbioru danych pasażerów *Titanica*) – ale, gdy zastąpimy je odpowiednio wartościami „0" i „1" – już tak (sposób przyporządkowania, oczywiście, jest tu dowolny). Z pomocą przychodzi nam tu klasa `LabelEncoder`[29] oraz jej metody – zresztą nie jedyna w podpakiecie *sklearn.preprocessing* (z innych skorzystamy później). Musimy najpierw utworzyć instancję tej klasy a następnie na jej rzecz użyć metody `fit_transform()`, której argumentem będzie kodowana kolumna danych:

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['sex'] = le.fit_transform(df['sex'])
df['sex'].unique()
```

```
[0, 1]
```

[26] Spróbuj odpowiedzieć na pytanie: dlaczego sułtan turecki nosił zielone szelki?

[27] Zjawisko to określa się mianem „przekleństwa wymiarowości": https://pl.wikipedia.org/wiki/Przekle%C5%84stwo_wymiarowo%C5%9Bci

[28] https://pandas.pydata.org/docs/reference/api/pandas.cut.html

[29] https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html

Jak widzimy, kolumna „sex” zawiera teraz zera i jedynki. Warto zauważyć, że – choć przekształcenie to powinno w zasadzie służyć do kodowania wartości zmiennej celu (czyli y), a nie zmiennych wejściowych (będziemy je oznaczali jako X) – to w wypadku kodowania binarnego możemy je zastosować bez przeszkód także do zmiennej uczącej.

Inaczej jednak postąpimy, gdy będzie ona przyjmowała wartości z więcej niż dwóch kategorii – wówczas kodowanie etykietowe wprowadzałoby nieuzasadniony (a nawet losowy) porządek wartości, którego przecież nie obserwujemy dla zmiennych kategorycznych – a nawet dla zmiennych porządkowych (takich jak np. wcześnie utworzona „age_cat”) chcielibyśmy mieć kontrolę nad wartościami numerycznymi, przypisywanymi kolejnym kategoriom[30]. Weźmy na początek na warsztat tę właśnie zmienną – chcielibyśmy, by kategorii „dziecko” była przypisana wartość 0, kategorii „nastolatek” – 1, itd., zgodnie z ustalonym przez nas porządkiem przedziałów.

Użyjemy klasy `OrdinalEncoder`[31], działającej podobnie jak LabelEncoder, ale pozwalającej – poprzez argument `categories` jej konstruktora – wyspecyfikować uporządkowaną listę kategorii, w kolejności których będą kodowanej zmiennej przypisywane kody numeryczne:

```
from sklearn.preprocessing import OrdinalEncoder
oe = OrdinalEncoder(categories=[['dziecko', 'nastolatek', 'dorosły',
                                 'staruszek']])
df['age_cat'] = oe.fit_transform(df[['age_cat']])
print(df['age_cat'].unique())
```

```
[2., 0., 3., 1.]
```

To, co wymaga szczególnej uwagi przy użyciu klasy `OrdinalEncoder` i jej metod to fakt, że są one dedykowane do kodowania wielu zmiennych uczących (wielu kolumn macierzy X) jednocześnie, a nie (tak, jak `LabelEncoder`) pojedynczej zmiennej celu (wektora y). W związku z tym – co widać w powyższym przykładzie – zarówno transformowane zmienne, jak i ich kategorie, muszą być obiektami dwuwymiarowymi: metoda `fit_transform()` oczekuje ramki danych (obiektu `DataFrame`), zawierającej kodowane kolumny, zaś argument `categories` konstruktora – listy, zawierającej jako swoje elementy listy kategorii dla każdej z kolumn owej ramki. Zwróć uwagę na podwójne nawiasy kwadratowe, definiujące listę list w konstruktorze oraz indeksowanie oryginalnej ramki danych `df` (jednoelementową w naszym wypadku) listą nazw kolumn.

Przejdźmy teraz do kodowania danych kategorycznych (ale bez ustalonego porządku), takich jak np. port zaokrętowania pasażera Titanica (kolumna „embarked”). Ponieważ zmienna ta przyjmuje więcej niż dwie wartości, to w tym wypadku użyjemy tzw. kodowania *one-hot*, zwanego też kodowaniem jeden z n, kodowaniem ze ślepą zmienną (ang. *dummy encoding*) lub mapą bitową (ang. *bitmap*). Jego idea polega na tym, że daną zmienną kodujemy za pomocą tylu dodatkowych kolumn, ile jest różnych wartości zmiennej kodowanej, nazywając te dodatkowe kolumny wartościami owej zmiennej (z ewentualnym dodaniem przed- lub przyrostka, danego nazwą kodowanej kolumny). Zatem dla naszego przykładu, skoro zmienna „embarked” przyjmuje wartości „C”, „Q” oraz „S”, kodując ją, dodamy do naszej ramki danych kolumny o nazwach „embarked_C”, „embarked_Q” oraz „embarked_S”. Jeżeli, dajmy na to, pewien pasażer na liście zaokrętował się w Cherbourgu („C”), to w odpowiadającym mu wierszu ramki danych w kolumnie „embarked_C” wstawimy jedynkę, zaś w kolumnach „embarked_Q” oraz „embarked_S” – zero. Innemu pasażerowi, który wsiadł na pokład w Queenstown („Q”), w odpowiadającym mu wierszu ramki danych w kolumnie „embarked_C” wstawimy zero, w kolumnie „embarked_Q” – jedynkę, zaś w kolumnie „embarked_S” – znów zero itd. W ten sposób każda wartość zmiennej kategorycznej będzie „równouprawniona” i nie wprowadzimy sztucznego ich uporządkowania, które mogłoby zaburzyć działanie naszego modelu uczenia maszynowego. Zobaczmy, jak takie

[30] Klasa `LabelEncoder` czyni to, przypisując wartości numeryczne w kolejności napotkanych w kolumnie wartości kodowanej zmiennej – co nie zawsze odpowiada kolejności etykiet zmiennej porządkowej.

[31] https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html

kodowanie zrealizować za pomocą klasy `OneHotEncoder`[32] z pakietu *sklearn.preprocessing*. Podobnie, jak poprzednio, utworzymy instancję tej klasy oraz użyjemy na jej rzecz metody `fit_transform()`:

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False)
ec = ohe.fit_transform(df[['embarked']])
cols = ohe.get_feature_names_out()
print(cols)
```

```
['embarked_C' 'embarked_Q' 'embarked_S']
```

Jak widać, metoda `get_feature_names_out()` klasy `OneHotEncoder` daje listę nazw kolumn kodujących – użyjemy jej, by dwuwymiarową tablicę `ec` pakietu *NumPy*, zwróconą przez metodę `fit_transform()`[33] przekształcić do ramki danych, którą z kolei dołączymy do pierwotnego zbioru danych pasażerów *Titanica*:

```
df = pd.concat([df, pd.DataFrame(ec, columns=cols)], axis='columns')
print(df.loc[[0, 10, 470],
             ['embarked', 'embarked_C', 'embarked_Q', 'embarked_S']])
```

```
    embarked   embarked_C   embarked_Q    embarked_S
0          S          0.0          0.0           1.0
10         C          1.0          0.0           0.0
470        Q          0.0          1.0           0.0
```

Wypisaliśmy wybrane wiersze, by jeszcze raz zilustrować zastosowany sposób kodowania. Niestety, sposób z użyciem klasy `OneHotEncoder` jest dość uciążliwy – na szczęście można tę czynność wykonać znacznie prościej, stosując funkcję `get_dummies()`[34] z pakietu *pandas*. Powoduje ona wprawdzie usunięcie oryginalnych, kodowanych kolumn danych[35], ale za to posiada szereg użytecznych argumentów, pozwalających wygodnie kontrolować proces kodowania. Sposób jej użycia jest wręcz banalny:

```
df = pd.get_dummies(df, columns=['embarked'])
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1309 entries, 0 to 1308
Data columns (total 16 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 [...]
 13  embarked_C  1309 non-null   uint8
 14  embarked_Q  1309 non-null   uint8
 15  embarked_S  1309 non-null   uint8
dtypes: float64(3), int64(4), object(6), uint8(3)
```

[32] https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html

[33] Gdybyśmy nie użyli argumentu `sparse_output` o wartości `False` w konstruktorze klasy, metoda ta zwróciłaby domyślnie tzw. macierz rzadką (ang. *sparse matrix*).

[34] https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html

[35] Można temu zapobiec, tworząc zawczasu ich kopie w pierwotnej ramce danych.

7. Podział na zbiór uczący i testowy

Aby model uczenia maszynowego miał dobrą zdolność predykcyjną, trzeba kontrolować sam proces uczenia, by nie doszło do tzw. przetrenowania (ang. *overfitting*) modelu. Sens tego pojęcia i sposoby zapobiegania mu omówimy nieco później – teraz przyjrzymy się temu, jak zapewnić sobie możliwość oceny, czy wytrenowany model będzie się dobrze sprawował w przyszłości na nowych, nie znanych mu danych. W tym celu z całego zestawu danych musimy wydzielić dwa podzbiory obserwacji – pierwszy, tzw. zbiór uczący (inaczej treningowy) będzie używany w procesie uczenia (ag. *fitting*) modelu. Drugi zaś, rozłączny z pierwszym, potraktujemy jako zbiór testowy, który posłuży nam właśnie do oszacowania „produkcyjnej" jakości modelu. By tego podziału dokonać, użyjemy funkcji `train_test_split()`[36] z pakietu *sklearn.model_selection*. Jej argumentami będą: macierz zmiennych uczących X[37], wektor zmiennej celu y oraz – alternatywnie – parametry `test_size` i `train_size`, określające proporcję liczby rekordów w odpowiednich zbiorach w stosunku do całości obserwacji.

Ponieważ chcielibyśmy, aby zbiór testowy jak najbardziej przypominał zbiór treningowy, na którym model będzie uczony, będziemy dążyli, by proporcje klas w obu zbiorach były zachowane[38]. Zazwyczaj więc, by spełnić ten warunek, użyjemy argumentu `stratify` o wartości równej wektorowi zmiennej celu y – wówczas w procesie losowania rekordów do odpowiednich podzbiorów[39] zostanie on użyty do wyliczenia proporcji klas wynikowych. Losowanie to możemy też kontrolować (np. by zapewnić jego powtarzalność) za pomocą argumentu `random_state`, którego wartość służy do zainicjowania ziarna (ang. *seed*) generatora liczb losowych:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    stratify=y)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(1047, 7) (262, 7) (1047,) (262,)
```

W wyniku działania funkcji otrzymaliśmy krotkę, zawierającą cztery zbiory danych, które podstawiliśmy pod odpowiednie zmienne. Rolę tych zbiorów pokazaliśmy, nadając zmiennym powszechnie stosowane nazwy. Warto nadmienić, że udział rekordów w zbiorze testowym powinien zależeć od liczby przypadków w całym zbiorze danych i być dobrany tak, by zbiór ten dobrze reprezentował różnorodność obserwacji ze świata rzeczywistego, na których nasz model będzie używany – jeżeli tych przypadków jest niewiele, udział ten może sięgnąć nawet 30%, zaś przy większej ich liczbie może wystarczyć 20, 10 a nawet kilka, czy wręcz jeden procent (gdy na przykład dysponujemy liczbą obserwacji rzędu miliona).

8. Skalowanie danych

Wiele metod uczenia maszynowego, które poznamy w kolejnych rozdziałach, bazuje na tzw. metryce odległości w wielowymiarowej przestrzeni zmiennych. Metody te są w związku z tym „uczulone" na zbiory danych, zawierające cechy, które mają różne zakresy, jednostki miary lub rzędy wielkości. W takich przypadkach różnice w rozpiętości cech mogą prowadzić do wypaczenia wydajności modelu lub trudności w procesie uczenia go. Dla osiągnięcia optymalnej wydajności będą one zatem wymagały

[36] https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

[37] Zakładamy, że wydzieliliśmy ją wcześniej z oryginalnego zbioru danych (podobnie, jak wektory), odrzucając odpowiednie kolumny (patrz rozdział „Wybór i inżynieria zmiennych").

[38] Oczywiście, dotyczy to uczenia nadzorowanego, a precyzyjniej mówiąc zadań klasyfikacji. W innych modelach predykcyjnych (np. dla regresji), czy w zadaniach uczenia nienadzorowanego, ten wymóg nie obowiązuje.

[39] Losowanie to działa „równolegle" dla macierzy X i wektora y – jeżeli wiersz macierzy X zostanie wybrany do np. zbioru testowego, trafi do niego także odpowiadający mu element wektora y.

procesu skalowania zmiennych. Stosujemy tu zazwyczaj dwa podejścia: standaryzację cech (ang. *standardization*) lub ich normalizację[40] minimaksową (ang. *normalization*).

Pierwsze z nich polega na takim przeskalowaniu istniejącej zmiennej, by nowa zmienna (nazwijmy ją z) miała średnią wartość równą zeru i jednostkowe odchylenie standardowe – a więc takie, jak dla rozkładu normalnego (Gaussa). Aby to osiągnąć, od każdego elementu x_i wektora skalowanej zmiennej należy odjąć jego średnią μ i podzielić przez jego odchylenie standardowe σ:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Metodę tę – zwaną też z-scoringiem, z powodu zwyczajowej nazwy nowej zmiennej – stosujemy zatem wtedy, gdy rozkład wartości skalowanej (pierwotnej) zmiennej jest zbliżony do normalnego. Jednocześnie radzi ona sobie dość dobrze z punktami oddalonymi. Spójrzmy, jak metoda ta zadziała dla zmiennej „fare" z zestawu danych pasażerów *Titanica*. Podobnie, jak to robiliśmy przy kodowaniu, musimy z pakietu *sklearn.preprocessing* zaimportować odpowiednią klasę (w tym wypadku `StandardScaler`[41]), utworzyć jej instancję, a następnie na jej rzecz wykonać metodę `fit_transform()`, której argumentem będzie ramka danych (lub dwuwymiarowa tablica *NumPy*), zawierająca skalowane zmienne[42]:

```
from sklearn.preprocessing import StandardScaler
ssc = StandardScaler()
df['fare_std'] = ssc.fit_transform(df[['fare']])
print(df['fare_std'].aggregate(['mean', 'std']))
```

```
mean  -8.691654e-17
std    1.000382e+00
Name: fare_std, dtype: float64
```

Otrzymane wartości – z dokładnością do precyzji reprezentacji liczb zmiennoprzecinkowych – odpowiadają zatem tym, które obserwujemy dla rozkładu normalnego. Warto dodać, że w pakiecie *sklearn.preprocessing* zawarta jest również podobnie działająca klasa o nazwie `RobustScaler`[43], jednak lepiej radząca sobie z punktami oddalonymi poprzez zastosowanie wyśrodkowania zmiennej do mediany (a nie średniej) i dzielenie przez znany nam już zakres międzykwartylowy (IQR), a nie odchylenie standardowe.

Kolejna klasa w pakiecie *sklearn.preprocessing*, używana do skalowania zmiennych, to `MinMaxScaler`. Wylicza ona wartość nowej zmiennej w taki sposób, by mieściła się ona w zadanym zakresie (domyślnie $[0, 1]$), poprzez zastosowanie następującego wzoru:

$$z_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

Zastosujmy tę metodę do zmiennej „age" z zestawu danych pasażerów *Titanica*:

```
from sklearn.preprocessing import MinMaxScaler
mmsc = MinMaxScaler()
df['age_std'] = mmsc.fit_transform(df[['age']])
print(df['age_std'].aggregate(['min', 'max']))
```

[40] Nie należy mylić tego podejścia z procesem tzw. normalizacji wektora, czyli dzieleniem każdego jego elementu przez normę (długość) wektora.

[41] https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html

[42] Klasa ta – podobnie, jak i następne, omawiane w tym rozdziale – została zaprojektowana do skalowania wielu zmiennych uczących (wielu kolumn macierzy X) jednocześnie.

[43] https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.RobustScaler.html

```
min   0.0
max   1.0
Name: age_std, dtype: float64
```

Niestety, metoda ta jest bardzo czuła na punkty oddalone – gdy takie znajdą się w kolumnie danych, następuje kompresja zakresu przeskalowanych wartości dla punktów „normalnych", co jest sytuacją niepożądaną. Inne metody transformacji zmiennych, dostępne w bibliotece *scikit-learn*, zostały omówione w jej dokumentacji[44].

Wszystkie omówione powyżej podejścia stosują liniową – a zatem w pełni odwracalną – transformację zmiennych. Możemy więc „odzyskać" ze zmiennej przeskalowanej zmienną oryginalną, stosując metodę `inverse_transform()` tych klas. Weźmy dla przykładu klasę `MinMaxScaler` i spróbujmy odtworzyć pierwotną wartość zmiennej „age":

```
df['age_inv'] = mmsc.inverse_transform(df[['age_std']])
print(df[['age', 'age_inv']].aggregate(['min', 'max']))
```

```
         age   age_inv
min   0.1667    0.1667
max   80.0000   80.0000
```

O ile metoda `fit_transform()` – będąca w istocie połączeniem metod `fit()` oraz `transform()` – znajduje najpierw parametry skalowania (odpowiednio: średnią i odchylenie standardowe, medianę i zakres IQR, czy minimum oraz maksimum), a następnie stosuje je do wybranych zmiennych, o tyle ta ostatnia korzysta z już wcześniej wyliczonych parametrów. Zobaczmy na przykładzie zmiennych „age" i „fare", jak ów proces przeprowadzić (zakładając, że dokonaliśmy podziału zbioru danych – patrz rozdział „Podział na zbiór uczący i testowy"):

```
from sklearn.preprocessing import RobustScaler
rsc = RobustScaler()
X_train_std = X_train.copy()
X_test_std = X_test.copy()
X_train_std[['age', 'fare']] = rsc.fit_transform(X_train[['age', 'fare']])
X_test_std[['age', 'fare']] = rsc.transform(X_test[['age', 'fare']])
print('Training set (scaled):')
print(X_train_std[['age', 'fare']].aggregate(['median', iqr]))
print('————————————————————————')
print('Test set (scaled):')
print(X_test_std[['age', 'fare']].aggregate(['median', iqr]))
```

```
Training set (scaled):
         age   fare
median   0.0    0.0
iqr      1.0    1.0
————————————————————————
Test set (scaled):
             age      fare
median   0.000000  -0.025325
iqr      0.847222   1.000625
```

Odnotujmy różnice w wartościach funkcji agregujących (mediany i IQR), wyliczonych dla zbioru treningowego i testowego.

[44] https://scikit-learn.org/stable/auto_examples/preprocessing/plot_all_scaling.html

9. Regresja liniowa

Modele regresji liniowej są dobrym punktem wyjścia dla zaprezentowania zadań regresji – czyli takich, gdzie zmienna celu (etykieta) y przyjmuje wartości rzeczywiste (ciągłe), a nie dyskretne, jak dla zadań klasyfikacji (tymi zajmiemy się później). Takie modele są popularne, ponieważ można je bardzo szybko wytrenować i są łatwo interpretowalne. Prawdopodobnie znasz najprostszą formę modelu regresji liniowej (tj. dopasowanie linii prostej do danych), określaną też mianem metody najmniejszych kwadratów, ale takie modele można rozszerzyć, aby modelować bardziej skomplikowane zachowanie danych.

W tym rozdziale zaczniemy od szybkiego, intuicyjnego omówienia matematyki stojącej za tym dobrze znanym problemem, a następnie pokażemy, jak można uogólniać modele liniowe, aby uwzględniały bardziej złożone wzorce w danych. W przypadku regresji liniowej (dla prostoty przyjmijmy na początek, że mamy do czynienia z jedną tylko zmienną niezależną x) nasz model będzie reprezentował dopasowanie prostej do zbioru punktów o współrzędnych (x, y), danej wzorem:

$$y = ax + b,$$

gdzie a jest współczynnikiem nachylenia (ang. *slope*) prostej do osi x, zaś b punktem jej przecięcia (ang. *intercept*) z osią y. Przykładowy zbiór danych (znajdziesz go w pliku *reglin_1.csv*) ma następującą postać:

```
x,y
3.745401188473625,3.2292689569426605
9.50714306409916,14.185654409388292
7.319939418114051,9.52423055383986
5.986584841970366,6.672065988351443
1.5601864044243652,-3.358149181518697
1.5599452033620265,-2.599953801670656
0.5808361216819946,-4.298966527595798
8.661761457749352,13.380645141717618
6.011150117432088,7.365918524432637
7.080725777960454,7.3984114005581745
...
```

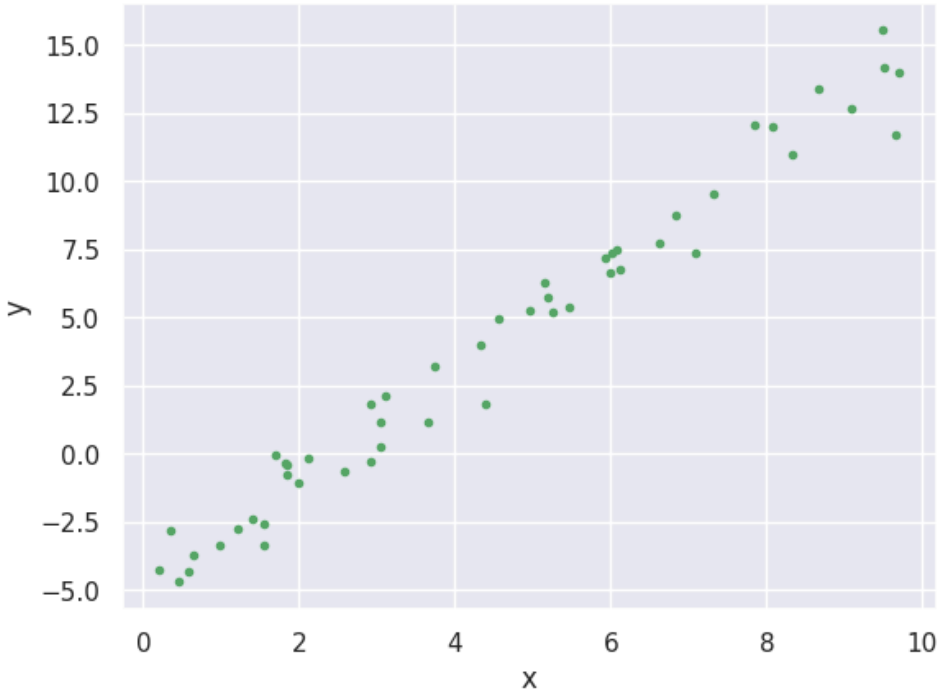
Na początek zaimportujemy niezbędne biblioteki:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

Następnie wczytamy przykładowy plik do ramki danych `df` i przyjrzymy się temu, jak punkty ze zbioru układają się na wykresie[45]:

```
df = pd.read_csv('reglin_1.csv')
fig = plt.figure()
axes = fig.gca()
axes.plot(df['x'], df['y'], 'g.')
axes.set_xlabel('x')
axes.set_ylabel('y');
```

[45] jest to tzw. wykres punktowy (ang. *scatter plot*)



Do skonstruowania linii prostej, najlepiej dopasowanej do tych danych, możemy wykorzystać estymator regresji liniowej[46] z pakietu *sklearn.linear_model*:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
X = df[['x']].values
y = df['y'].values
model.fit(X, y);
```

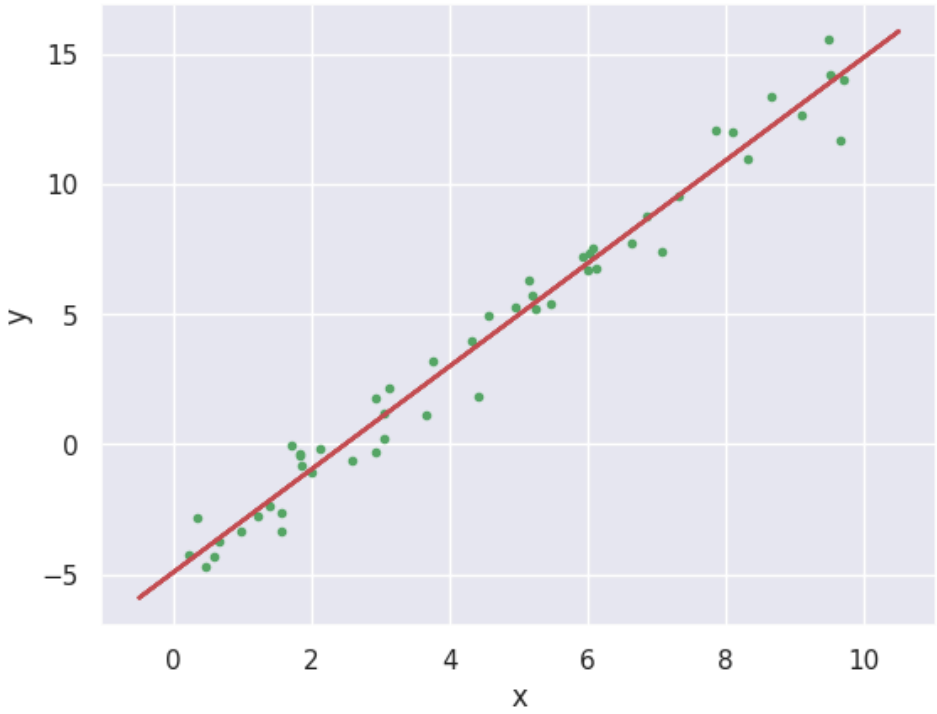
Jak widać, mamy tu typowy i powtarzalny schemat postępowania: najpierw zaimportowanie odpowiedniej klasy (reprezentującej model) z właściwego podpakietu biblioteki *scikit-learn* (w tym wypadku modeli liniowych), następnie utworzenie instancji tej klasy (z ewentualnym wyborem hiperparametrów modelu[47], jeżeli miałyby one być inne, niż domyślne), wreszcie użycie metody `fit()`, której argumentami jest macierz zmiennych niezależnych oraz wektor zmiennej celu y. Zwróć uwagę, że pierwszy argument metody `fit()` musi być obiektem dwuwymiarowym (podwójne nawiasy kwadratowe w wyrażeniu `X = df[['x']].values`).

Kolejnym krokiem jest predykcja, której dokonujemy za pomocą metody `predict()` – ona także wymaga dwuwymiarowej tablicy jako swojego argumentu – stąd argument `np.newaxis`, służący przekształceniu jednowymiarowego wektora `x_fit` w macierz. Najpierw jednak skonstruujemy za pomocą funkcji `linspace()` zbiór 500 wartości, równomiernie rozłożonych na przedziale $[-0.5, 10.5]$. Mając przewidziane przez model wartości zmiennej celu dla tych punktów, możemy na nasz poprzedni wykres nanieść linię prostą, obrazującą dopasowanie, dane przez model:

```
x_fit = np.linspace(-0.5, 10.5, 500)
y_fit = model.predict(x_fit[:, np.newaxis])
axes.plot(x_fit, y_fit, 'r-')
fig
```

[46] https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

[47] Są to parametry, charakteryzujące model, ustalane przed procesem uczenia – decydują one o wyborze konkretnego modelu z całej ich klasy, mogą też posłużyć określeniu stopnia złożoności modelu.



Współczynnik nachylenia prostej i punkt jej przecięcia z osią y są zawarte w atrybutach dopasowanego modelu, które w pakiecie *scikit-learn* są zawsze oznaczone końcowym podkreślnikiem. W naszym wypadku odpowiednimi parametrami są `coef_` (jest to wektor o liczbie elementów, równej liczbie zmiennych uczących) i `intercept_`:

```
print('a = ', model.coef_[0])
print('b = ', model.intercept_)
```

```
a =  1.9776566003853107
b =  -4.903310725531115
```

Jednakże estymator `LinearRegression` potrafi znacznie więcej – oprócz dopasowania linii prostej w przypadku jednej zmiennej uczącej x (problem jednowymiarowy), może również operować na wielowymiarowych modelach liniowych postaci:

$$y = b + a_1 x_1 + a_2 x_2 + ... + a_n x_n,$$

gdzie występuje wiele zmiennych niezależnych x_i. Geometrycznie jest to równoważne np. w trzech wymiarach dopasowaniu płaszczyzny do punktów w przestrzeni (a_1 i a_2 będą w tym wypadku współczynnikami jej nachylenia odpowiednio do osi x_1 i x_2) lub w ogólności dopasowaniu n-wymiarowej hiperpłaszczyzny do punktów, leżącej w hiperprzestrzeni o $n + 1$ wymiarach.

Sztuczką, której można użyć, aby dostosować regresję liniową do nieliniowych relacji między zmiennymi, jest transformacja danych za pomocą tzw. funkcji bazowych. Pomysł polega na wzięciu dotychczasowego wielowymiarowego modelu liniowego i utworzeniu w sposób sztuczny nowych zmiennych x_1, x_2, x_3, itd. z naszego jednowymiarowego wektora wejściowego x. Oznacza to, że kładziemy $x_i = f_i(x)$, gdzie f_i jest pewną funkcją, która transformuje nasze pierwotne dane. Na przykład, jeśli $f_i(x) = x^i$, nasz model staje się regresją wielomianową:

$$y = b + a_1 x + a_2 x^2 + a_3 x^3 + ... + a_n x^n.$$

Zauważ, że jest to nadal model liniowy – pojęcie to odnosi się do faktu, że współczynniki a_i nigdy się nie mnożą ani nie dzielą. To, co w rzeczywistości zrobiliśmy, to wzięcie naszych jednowymiarowych wartości x i rzutowanie ich do wyższego wymiaru, tak aby dopasowanie liniowe mogło odwzorowywać bardziej skomplikowane relacje między zmiennymi x oraz y. Oczywiście, możliwe do zastosowania są też inne funkcje bazowe. Na przykład, jednym z użytecznych wzorców jest zbudowanie modelu, w którym funkcja odwzorowująca x na y nie jest sumą bazowych funkcji wielomianowych, ale funkcji Gaussa, z różnymi wartościami średnimi i odchyleniami standardowymi rozkładu normalnego.

Jak ocenić jakość naszego modelu? Wspominaliśmy już, że metoda regresji liniowej zwana jest też metodą najmniejszych kwadratów. Określenie to bierze się stąd, że tak skonstruowany model minimalizuje sumę kwadratów (drugich potęg) odległości punktów o współrzędnych (x, y) od prostej regresji[48]. Suma ta jest jedną z tzw. funkcji oceny (ang. *scoring function*), zwanej też metryką wydajności (ang. *performance metrics*) modelu. Dla łatwiejszej jej interpretacji dzieli się zazwyczaj tę sumę przez liczbę obserwacji, nazywając tak otrzymaną wartość średnim błędem kwadratowym (ang. *mean squared error* – MSE) – a ponieważ jej miano jest kwadratem miana zmiennej y[49], dla bezpośredniego porównania błędu dopasowania z wartością zmiennej celu, pierwiastkujemy tę metrykę (ang. *root mean squared error* – RMSE). Inną, często stosowaną metryką, jest tzw. współczynnik determinacji[50] R^2 – im bliższa jedności wartość tego współczynnika, tym lepsze dopasowanie modelu. Funkcje te zawarte są w podpakiecie *sklearn.metrics* – wystarczy więc z nich po prostu odpowiednio skorzystać, uprzednio wyliczywszy wartości przewidziane przez model dla danych wartości x:

```
from sklearn.metrics import mean_squared_error, root_mean_squared_error, r2_score
y_pred = model.predict(X)
print('MSE  = ', mean_squared_error(y, y_pred))
print('RMSE = ', root_mean_squared_error(y, y_pred))
print('R2   = ', r2_score(y, y_pred))
```

```
MSE  =  0.8230711437486881
RMSE =  0.9072326844579003
R2   =  0.9749140085676858
```

10. Klasyfikacja

W tym rozdziale przyjrzymy się bliżej wybranym metodom klasyfikacji – naiwnemu klasyfikatorowi Bayesa oraz maszynie wektorów nośnych. Założymy na początek, że nasza zmienna celu y ma charakter binarny (tak, jak w przykładzie opisującym pasażerów *Titanica*). Zazwyczaj przykłady uczące, ze zmienną celu o wartości 0 lub `False` określamy mianem **klasy negatywnej**, zaś te z wartościami 1 lub `True` – **klasy pozytywnej**. Późniejsze uogólnienie zaprezentowanych tu pojęć i technik będzie stosunkowo proste.

Zbiór danych, którym będziemy się posługiwać w poniższych przykładach, zostanie sztucznie wygenerowany w postaci dwóch odrębnych grup punktów (opisanych zmiennymi x_1 i x_2), należących do dwóch różnych klas – użyjemy w tym celu funkcji `make_blobs()` z pakietu *sklearn.datasets*[51]. Oczywiście, ma on charakter wyłącznie ilustracyjny – w rzeczywistych sytuacjach „z życia wziętych" zazwyczaj będziemy mieli do czynienia z bardziej skomplikowanymi zależnościami, zwłaszcza, jeżeli chodzi o liczbę zmiennych wejściowych, czy klas zmiennej celu, ich separację itd. Takie przykłady pojawią się więc w dalszych ćwiczeniach, które znajdziesz w kolejnych częściach niniejszego skryptu.

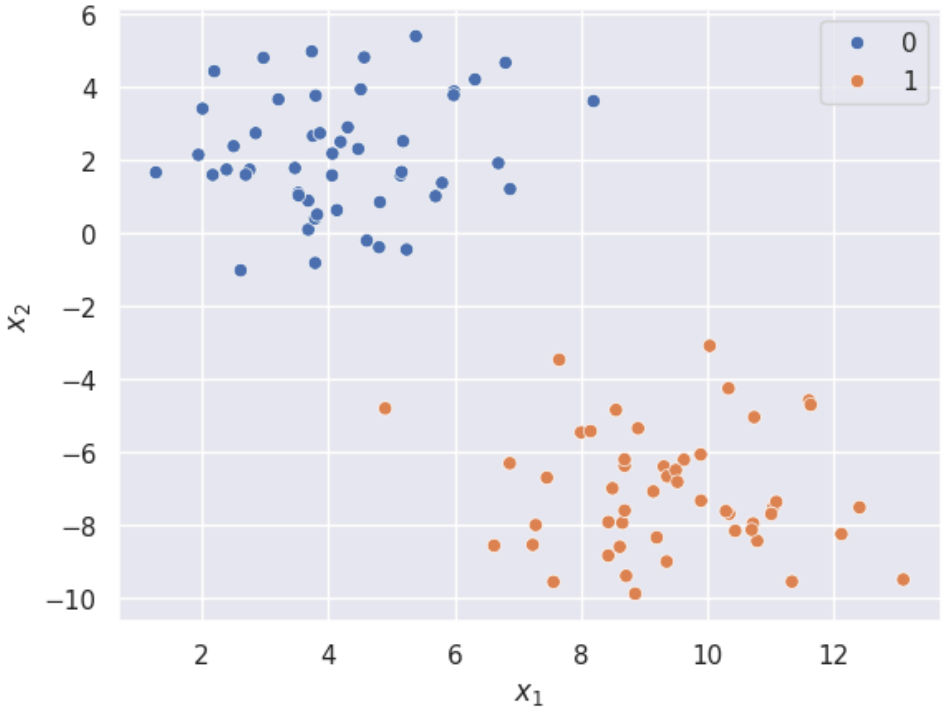
[48] a w ogólności od hiperpłaszczyzny regresji

[49] Gdyby była ona dana np. w metrach, MSE wyliczylibyśmy w metrach kwadratowych.

[50] https://pl.wikipedia.org/wiki/Wsp%C3%B3%C5%82czynnik_determinacji – można go również wyznaczyć za pomocą metody `score()` klasy modelu

[51] https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_blobs.html

```
from sklearn.datasets import make_blobs
X, y = make_blobs(100, 2, centers=2, cluster_std=1.5, random_state=36)
sns.scatterplot(x=X[:, 0], y=X[:, 1], hue=y)
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.legend(loc='upper left');
```



10.1. Naiwny klasyfikator Bayesa

Naiwne modele Bayesa to grupa niezwykle szybkich i prostych algorytmów klasyfikacyjnych, które często nadają się do wysokowymiarowych (tj. posiadających dużą liczbę zmiennych) zestawów danych. Ze względu właśnie na ich efektywność i małą liczbę dostrajalnych hiperparametrów, okazują się bardzo przydatne jako naprędce przygotowane modele bazowe dla problemu klasyfikacji. W tym podrozdziale skupimy się na intuicyjnym wyjaśnieniu, jak one działają, opierając się na wcześniej opisanym, przykładowym zestawie danych.

Naiwne klasyfikatory Bayesa bazują na tzw. twierdzeniu Bayesa[52], które jest równaniem opisującym związek **prawdopodobieństw warunkowych** zmiennych statystycznych. W klasyfikacji bayesowskiej interesuje nas znalezienie prawdopodobieństwa etykiety L przy danych zaobserwowanych cechach, które możemy zapisać jako $P(L|\text{cechy})$. Twierdzenie Bayesa mówi nam, jak wyrazić to prawdopodobieństwo poprzez wielkości, które możemy obliczyć bardziej bezpośrednio:

$$P(L|\text{cechy}) = \frac{P(\text{cechy}|L)P(L)}{P(\text{cechy})}.$$

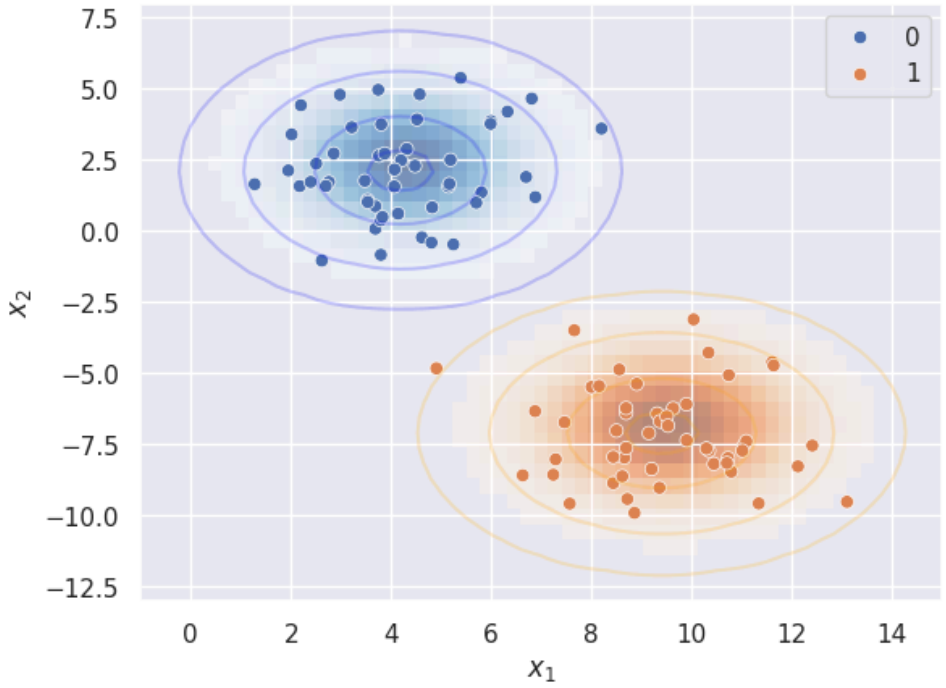
Jeśli próbujemy rozróżnić pomiędzy dwiema etykietami – nazwijmy je L_1 i L_2 – to jednym ze sposobów podjęcia tej decyzji jest obliczenie stosunku prawdopodobieństw *a posteriori* dla każdej etykiety, a następnie wybranie jako predykcję modelu klasy, odpowiadającej tej etykiecie, dla której owo prawdopodobieństwo jest największe:

[52] https://pl.wikipedia.org/wiki/Twierdzenie_Bayesa

$$\frac{P(L_1|\text{cechy})}{P(L_2|\text{cechy})} = \frac{P(\text{cechy}|L_1)P(L_1)}{P(\text{cechy}|L_2)P(L_2)}.$$

Wszystko, czego teraz potrzebujemy, to model, za pomocą którego możemy obliczyć $P(\text{cechy}|L_i)$ dla każdej etykiety. Taki model nazywa się **modelem generatywnym**, ponieważ określa hipotetyczny proces losowy, który generuje dane. Znalezienie go dla każdej wartości etykiety jest głównym zadaniem w procesie trenowania klasyfikatora bayesowskiego. Jest ono w ogólności bardzo trudne, ale możemy je uprościć, przyjmując pewne założenia dotyczące formy tegoż modelu – stąd słowo „naiwny" w nazwach tej grupy metod.

Chyba najłatwiejszym do zrozumienia naiwnym klasyfikatorem Bayesa jest klasyfikator gaussowski. Przyjmujemy tu założenie, że dane x_i dla każdej etykiety są losowo wybierane z prostego rozkładu normalnego (Gaussa). Zobaczmy, jak takie założenie działa dla naszego przykładowego zbioru danych. Przyjmując zerową kowariancję pomiędzy zmiennymi x_1 i x_2 (a wiec traktując je jako zmienne zupełnie niezależne – co jest kolejną „naiwnością" modelu), możemy łatwo dopasować wartość średnią i odchylenie standardowe każdej ze zmiennych dla zbioru punktów w obrębie każdej etykiety – a to wszystko, czego potrzebujemy, by zdefiniować potrzebny nam rozkład. Wynik tego dopasowania pokazano na poniższym rysunku:



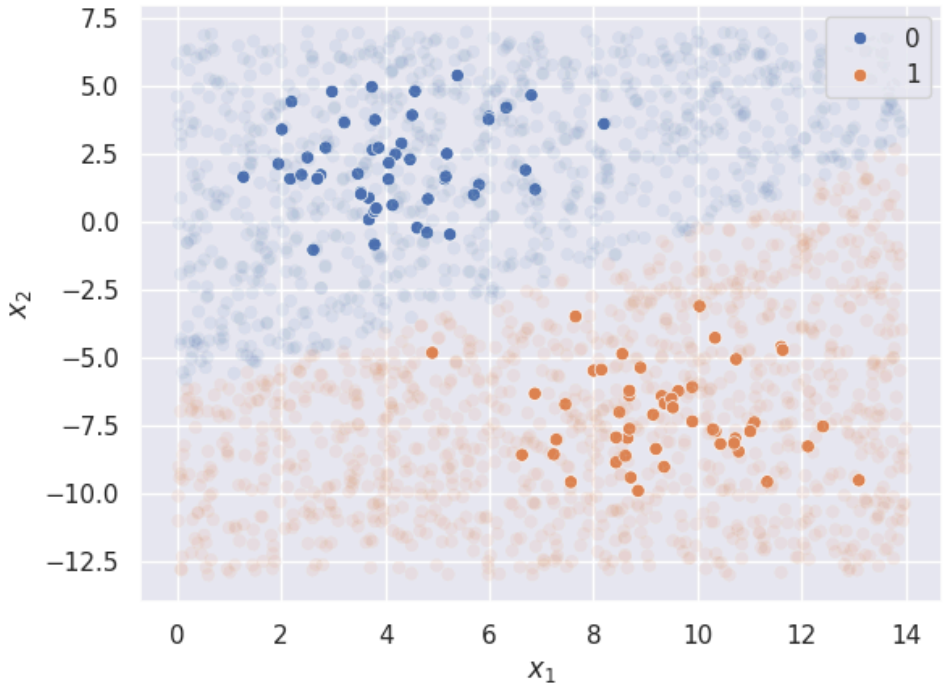
Elipsy na powyższym rysunku reprezentują model generatywny Gaussa dla każdej etykiety, z prawdopodobieństwem rosnącym w kierunku środka elips. Dzięki temu modelowi, wyznaczonemu dla każdej klasy, mamy prosty przepis na obliczenie prawdopodobieństwa $P(\text{cechy}|L_i)$ dla dowolnego punktu danych – a to pozwala nam szybko obliczyć stosunek prawdopodobieństw *a posteriori* i określić, która etykieta jest najbardziej prawdopodobna dla danego punktu. Procedura ta została zaimplementowana w estymatorze `GaussianNB`[53], zawartym w pakiecie *sklearn.naive_bayes*:

```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X, y);
```

[53] https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html

Wygenerujmy teraz losowo nowe dane i przewidźmy dla nich etykietę, umieszczając je na rysunku (stosując półprzezroczystość, by odróżnić je od punktów zbioru uczącego, ale zachowując oryginalny kolor), aby zorientować się, gdzie przebiega tzw. granica decyzyjna:

```
rng = np.random.RandomState(0)
X_new = [0, -13] + [14, 20] * rng.rand(2000, 2)
y_new = model.predict(X_new)
sns.scatterplot(x=X[:, 0], y=X[:, 1], hue=y)
sns.scatterplot(x=X_new[:, 0], y=X_new[:, 1], hue=y_new, legend=False, alpha=0.1)
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.legend(loc='upper left');
```



Przydatnym elementem formalizmu bayesowskiego jest to, że w sposób naturalny pozwala on na klasyfikację probabilistyczną, którą możemy wyznaczyć za pomocą metody `predict_proba()`:

```
y_prob = model.predict_proba(X_new)
y_prob[-8:].round(2)
```

```
array([[1.  , 0.  ],
       [0.  , 1.  ],
       [0.  , 1.  ],
       [0.  , 1.  ],
       [0.43, 0.57],
       [0.08, 0.92],
       [1.  , 0.  ],
       [0.  , 1.  ]])
```

Kolumny powyższej macierzy dają odpowiednio prawdopodobieństwa *a posteriori* przynależności danego przypadku do klasy negatywnej i pozytywnej. Jeśli zależy nam na oszacowaniu niepewności klasyfikacji, podejście bayesowskie, takie jak niniejsze, może okazać się użyteczne. W ogólności jednak, nie zawsze założenie o niezależności zmiennych uczących jest prawdziwe i nie zawsze ich rozkład

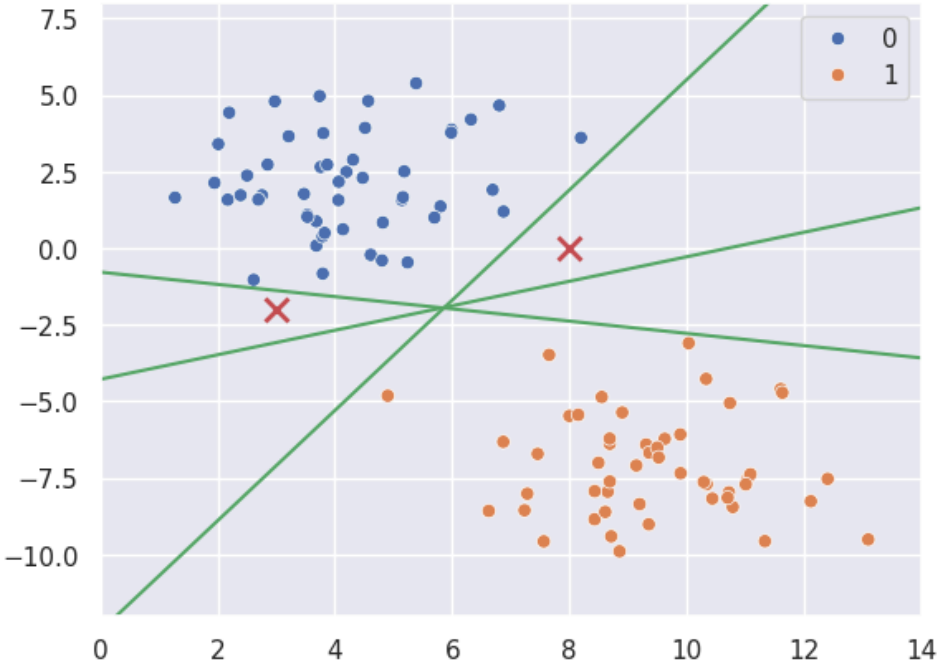
statystyczny ma charakter gaussowski. Stąd metoda ta często nie daje zbyt dobrych rezultatów. Mimo to w wielu przypadkach – zwłaszcza gdy liczba cech staje się duża – poczynione założenia nie są na tyle ograniczające, aby uniemożliwić naiwnemu klasyfikatorowi Bayesa bycie użyteczną metodą.

10.2. Maszyny wektorów nośnych

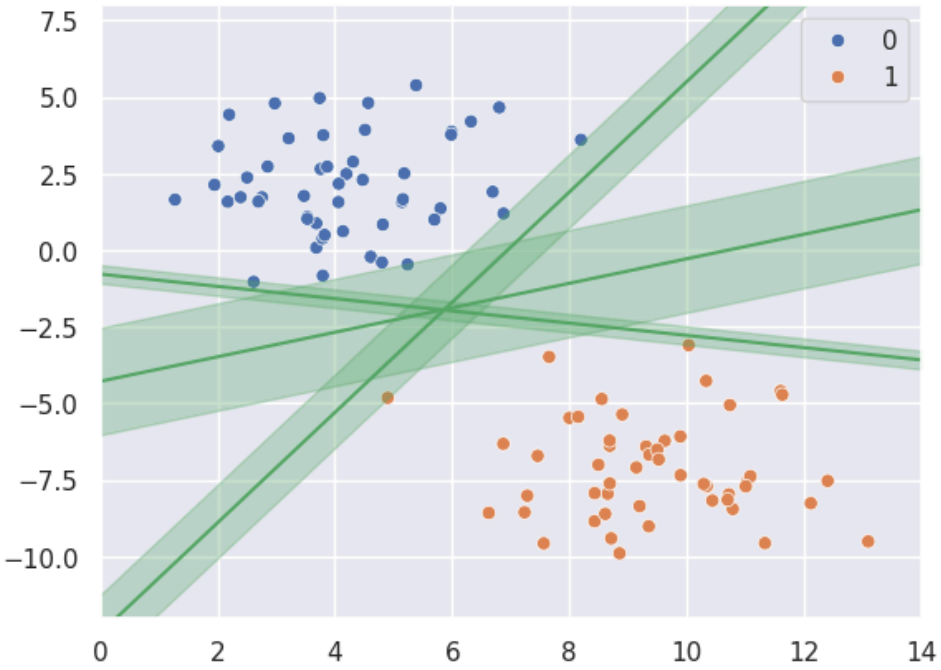
Maszyny wektorów nośnych (ang. *Support Vector Machines* – SVM) to szczególnie potężna i elastyczna klasa nadzorowanych algorytmów uczenia maszynowego, mających zastosowanie zarówno do klasyfikacji, jak i regresji. W tym podrozdziale w sposób intuicyjny rozwiniemy zasady matematyczne, stojące za maszynami wektorów nośnych i ich zastosowaniem w problemach klasyfikacji.

W ramach dyskusji na temat klasyfikacji bayesowskiej poznaliśmy prosty model, opisujący rozkład statystyczny zmiennych dla każdej klasy i użyliśmy modeli statystycznych do probabilistycznego określania etykiet dla nowych punktów. Był to przykład klasyfikacji generatywnej – teraz rozważymy **klasyfikację dyskryminacyjną**: zamiast modelować każdą klasę, po prostu znajdziemy linię (w dwóch wymiarach) bądź hiperpowierzchnię (w wielu wymiarach), która dzieli klasy od siebie.

Jako przykład weźmy znany już nam zbiór wygenerowanych wcześniej punktów, dla którego separacja klas jest wyraźnie widoczna. Liniowy klasyfikator dyskryminacyjny miałby za zadanie znaleźć linię prostą, oddzielającą dwa podzbiory danych (należące do różnych klas) i w ten sposób stworzyć model klasyfikacji. W przypadku dwuwymiarowych danych, takich jak nasze, jest to zadanie, które moglibyśmy wykonać ręcznie. Ale od razu widzimy problem: istnieje więcej niż jedna możliwa linia podziału, która może idealnie rozróżnić dwie klasy:



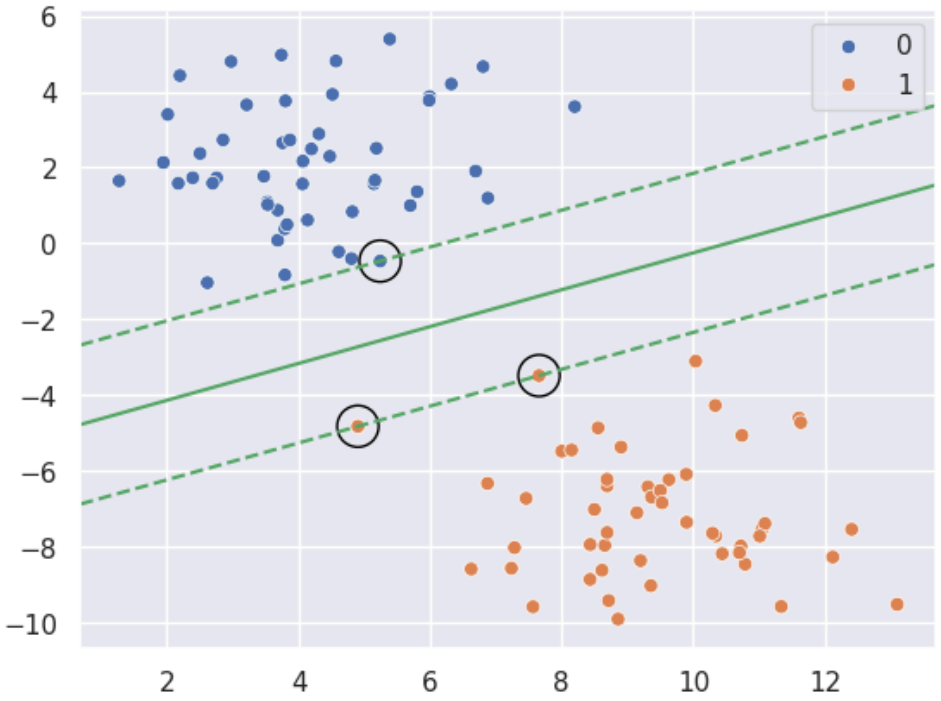
Są to trzy zdecydowanie różne separatory, które jednak doskonale rozróżniają przypadki różnych klas. W zależności od tego, który wybierzemy, nowe punkty danych (np. takie, jak oznaczone symbolem „X" na powyższym rysunku) zostaną oznaczone inną etykietą. Najwyraźniej nasza prosta intuicja „rysowania linii między klasami" nie wystarczy i musimy przemyśleć to zagadnienie nieco głębiej. Maszyny wektorów nośnych oferują – oparty o ścisłe rozważania matematyczne – sposób na wzmocnienie naszej intuicji: zamiast po prostu rysować linię o zerowej szerokości między klasami, możemy narysować wokół każdej linii **margines** o pewnej szerokości, aż do najbliższego punktu danych, należącego do jednej bądź drugiej klasy. W maszynach wektorów nośnych to właśnie linia maksymalizująca ów margines jest tą, którą wybierzemy jako reprezentującą optymalny model. Na poniższym rysunku widać, jak mogłoby to wyglądać w naszym przypadku:



Zobaczmy wynik rzeczywistego dopasowania do tych danych, z użyciem klasyfikatora opartego o powyższą ideę z pakietu *scikit-learn*. Na początek użyjemy tzw. liniowego jądra (ang. *kernel*) i ustawimy hiperparametr `C` modelu na bardzo dużą liczbę (omówimy znaczenie tych wartości bardziej szczegółowo za chwilę):

```
from sklearn.svm import SVC
model = SVC(kernel='linear', C=1e6)
model.fit(X, y);
```

Oto, jak wygląda granica decyzyjna, dana przez wytrenowany model, wraz z marginesem separacji klas – maksymalnym, możliwym do uzyskania:



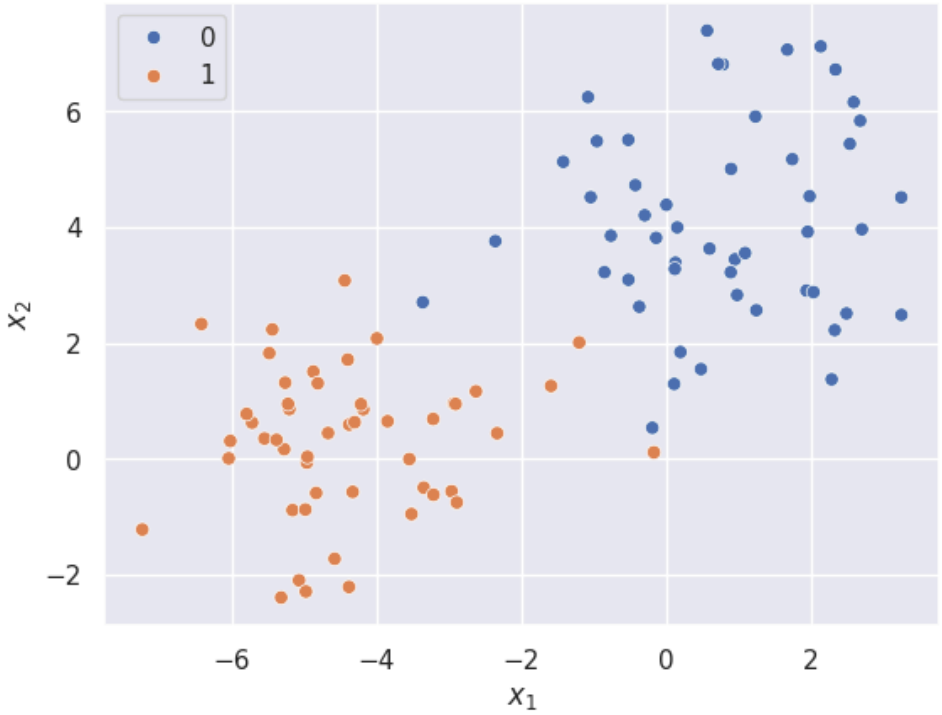
Zauważ, że kilka punktów treningowych dotyka marginesu: są one oznaczone czarnymi kółkami na powyższym rysunku. Punkty te – określane mianem **wektorów nośnych** (stąd nazwa algorytmu) – są kluczowymi elementami tego dopasowania. W pakiecie *scikit-learn* współrzędne tych punktów są przechowywane w atrybucie `support_vectors_` obiektu klasyfikatora:

```
model.support_vectors_
```

```
array([[ 5.24143075, -0.46227863],
       [ 4.90299279, -4.81646751],
       [ 7.65554732, -3.48131134]])
```

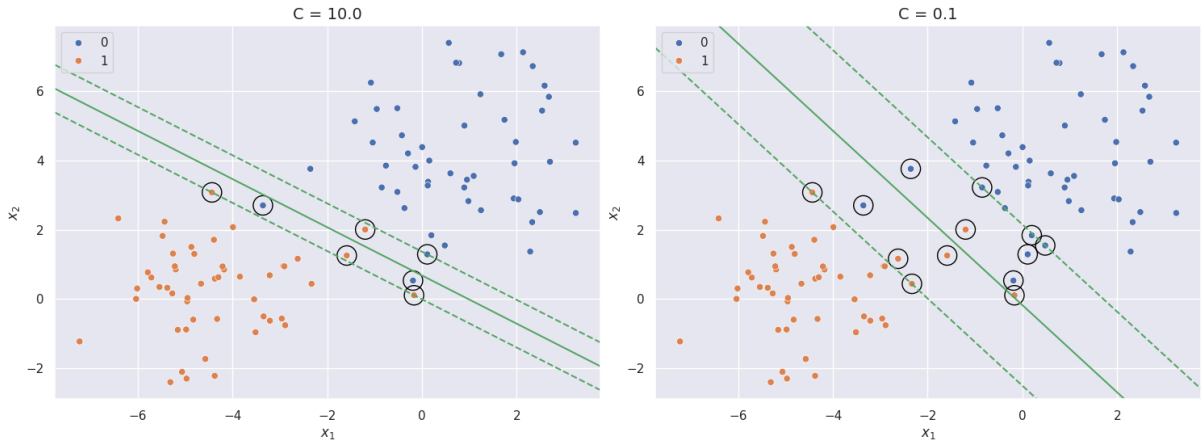
Kluczem do sukcesu klasyfikatora SVM jest to, że dla dopasowania liczy się tylko położenie wektorów nośnych; punkty dalej od marginesu, które znajdują się po właściwej stronie, nie modyfikują go. Technicznie rzecz biorąc, dzieje się tak, ponieważ punkty te nie wnoszą wkładu do funkcji straty używanej do trenowania modelu, więc ich położenie i liczba nie mają znaczenia, dopóki nie przekraczają marginesu.

Nasza dyskusja do tej pory koncentrowała się wokół bardzo czystych zestawów danych, w których istnieje idealna granica decyzyjna. Ale co, jeśli (jak to często bywa w rzeczywistości) dane mają pewną liczbę punktów, które już na pierwszy rzut oka trudno przyporządkować do którejś z klas i narysowanie granicy podziału w formie linii prostej jest niemożliwe. Weźmy na przykład taki zbiór danych[54]:



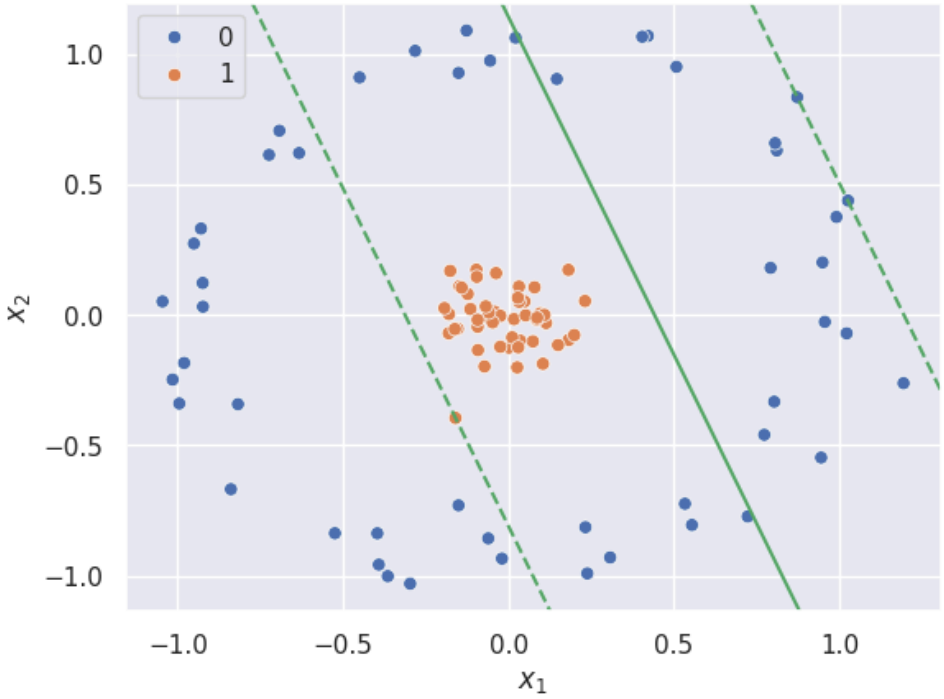
Aby poradzić sobie z tym przypadkiem, w algorytmie SVM zaimplementowano tzw. **zmienne rozluźniające** (ang. *slack variables*), którw „zmiękczają" margines: oznacza to, że pozwalamy niektórym punktom „wpełznąć" do marginesu, jeśli pozwali to na lepsze dopasowanie modelu. Twardość marginesu jest kontrolowana przez hiperparametr, najczęściej znany jako `C`: dla bardzo dużych jego wartości margines jest twardy i punkty nie mogą się w nim znajdować. Dla mniejszego `C` margines jest miększy i może objąć niektóre punkty. Poniższy wykres daje obraz tego, jak zmieniający się hiperparametr `C` (przyjmuje on odpowiednio wartości 10 i 0.1 dla lewego i prawego rysunku) wpływa na ostateczne dopasowanie modelu, poprzez zmiękczenie marginesu:

[54] Możesz go wygenerować, kładąc `random_state=3` jako argument funkcji `make_blobs()` w kodzie, zaprezentowanym na początku tego rozdziału.



Optymalna wartość hiperparametru `C` będzie zależeć od zestawu danych i powinna zostać dostrojona za pomocą walidacji krzyżowej lub podobnej procedury – zagadnienia te zostaną omówione w kolejnych kursach.

Algorytm SVM staje się niezwykle potężny, gdy jest połączony z transformacją oryginalnych zmiennych uczących do wyżej wymiarowej przestrzeni dzięki tzw. **jądru przekształcenia**. Spotkaliśmy się już z takim podejściem przy okazji regresji za pomocą nieliniowych funkcji bazowych (patrz Rozdział 9) – tam rzutowaliśmy nasze dane na przestrzeń o wyższym wymiarze zdefiniowaną przez wielomiany i funkcje bazowe Gaussa, dzięki czemu byliśmy w stanie dopasować nieliniowe relacje za pomocą klasyfikatora liniowego. W modelach SVM możemy użyć analogicznego pomysłu – aby uzasadnić potrzebę użycia jąder, przyjrzyjmy się pewnym danym, które nie są liniowo separowalne:

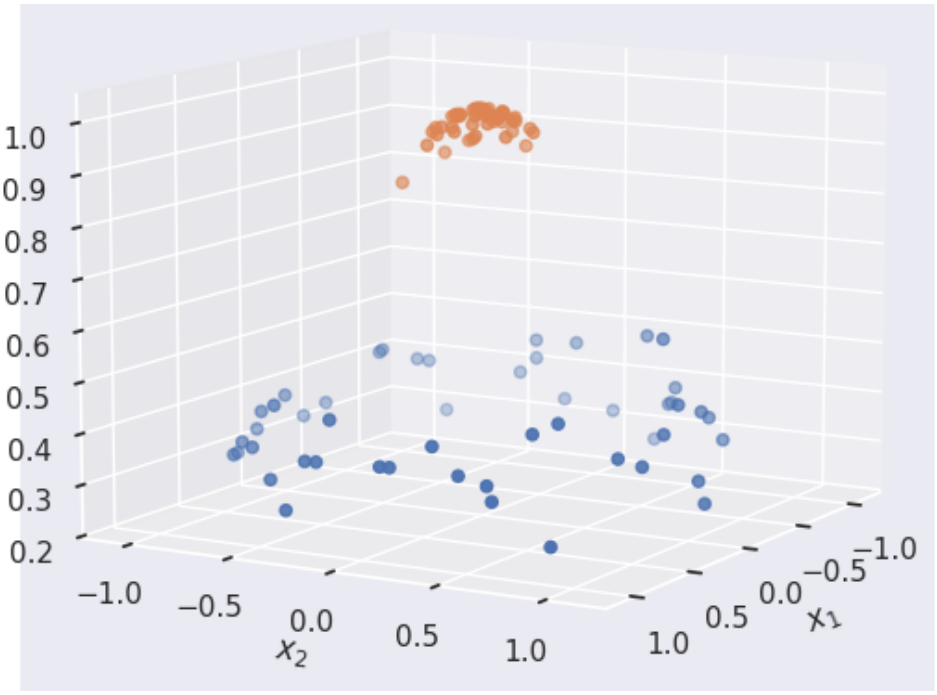


Jest oczywiste, że żadna liniowa dyskryminacja za pomocą prostej nigdy nie będzie w stanie rozdzielić tych danych na dwie czyste klasy. Możemy jednak zastanowić się, jak moglibyśmy rzutować dane do wyższego wymiaru (za pomocą funkcji nieliniowej), tak aby liniowy separator był wystarczający. Na przykład, prostą i często stosowaną projekcją, którą moglibyśmy zastosować, byłoby użycie jako jądra przekształcenia tzw. radialnej funkcji bazowej (ang. *radial basis function* – RBF)[55]:

[55]Współczynnik γ jest hiperparametrem algorytmu, stosowanym dla jąder wielomianowych.

$$K(x, x') = \exp(-\gamma \, \|x - x'\|^2)$$

Zwizualizujmy efekt jej działania w postaci wykresu trójwymiarowego (na osi pionowej mamy nową zmienną, wprowadzoną przez wybranie szczególnej formy powyższego przekształcenia, pozwalającej na wyodrębnienie zbioru punktów w centrum wykresu):



Widzimy, że dzięki dodatkowemu wymiarowi dane stają się liniowo separowalne w sposób trywialny, poprzez narysowanie płaszczyzny rozdzielającej, na przykład dla wartości nowej zmiennej równej 0.6. W naszym wypadku musieliśmy jednak odpowiednio wybrać i precyzyjnie dostroić naszą projekcję: gdybyśmy nie wyśrodkowali naszej radialnej funkcji bazowej w odpowiednim miejscu, nie zobaczylibyśmy tak czystych, liniowo separowalnych zbiorów punktów. Wybór ten, w ogólności, stanowi zasadniczy problem: chcielibyśmy w jakiś sposób automatycznie znaleźć optymalne funkcje bazowe.

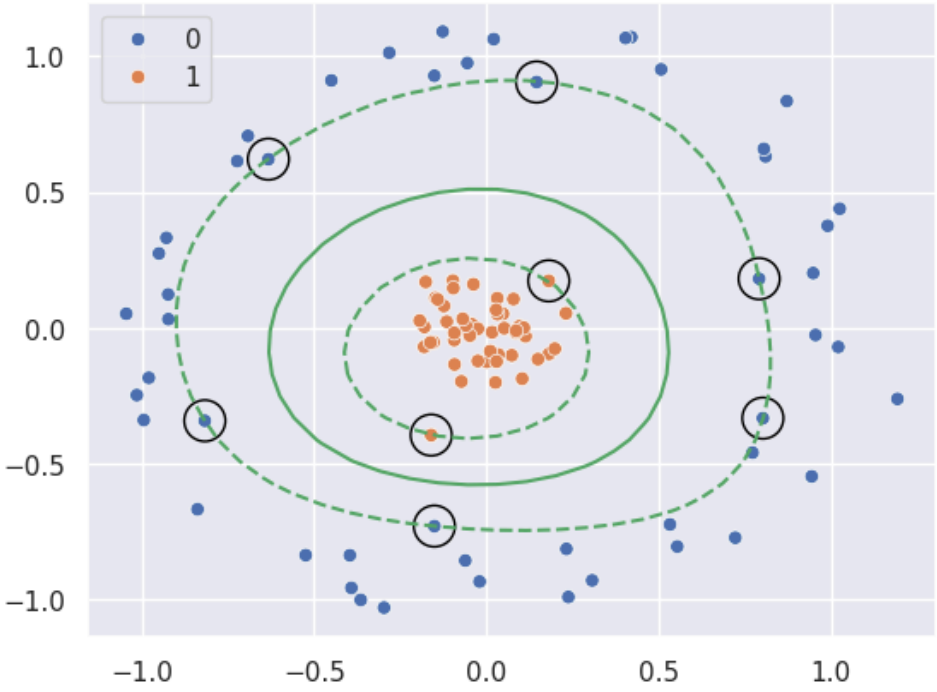
Jedną ze strategii, stosowanych w tym celu, jest obliczenie funkcji bazowej wyśrodkowanej w każdym punkcie zestawu danych i pozwolenie algorytmowi SVM na przejrzenie wyników i znalezienie optymalnego rozwiązania. Ten typ transformacji funkcji bazowej jest znany jako transformacja za pomocą jądra przekształcenia, ponieważ opiera się na relacji podobieństwa między każdą parą punktów. Potencjalnym problemem tej strategii jest to, że może stać się bardzo wymagający obliczeniowo, gdy liczba zmiennych rośnie. Jednak dzięki sprytnej procedurze, znanej jako **sztuczka z jądrem** (ang. *kernel trick*), dopasowanie danych przekształconych przez jądro może być wykonane niejawnie – to znaczy, bez budowania pełnej wysokowymiarowej reprezentacji w przestrzeni nowych, wyrzutowanych zmiennych. Procedura ta jest wbudowana w SVM i jest jednym z powodów, dla których metoda jest tak efektywna i chętnie stosowana w uczeniu maszynowym, aby przekształcić szybkie metody liniowe w szybkie metody nieliniowe, szczególnie w przypadku modeli, w których można użyć sztuczki z jądrem. Tu wystarczy, że zmienimy wartość argumentu `kernel` w konstruktorze klasy `SVC` z `linear` na `rbf`[56]:

```
model = SVC(kernel='rbf', C=1e6)
model.fit(X, y);
```

Przyjęliśmy, dla prostoty, domyślną wartość hiperparametru `gamma` – jednak podobnie, jak dla hiperparametru `C` należy go dostroić, w zależności od zestawu danych.

[56] Jest to kolejny (poza `C` i `gamma`) hiperparametr tego algorytmu – inne możliwe wartości to np. `poly` lub `sigmoid` (patrz https://scikit-learn.org/stable/auto_examples/svm/plot_svm_kernels.html).

Zobaczmy, jaki kształt przyjęła teraz granica decyzyjna oraz margines separacji klas – jak widać, nie ma ona postaci linii prostej a nieliniowej krzywej zamkniętej, dzięki czemu separacji klas udało się dokonać w sposób idealny.



10.3. Ocena jakości klasyfikacji

Wróćmy na chwilę do zbioru danych, zawierającego dwa skupiska punktów, losowo wygenerowane z wartością ziarna generatora liczb losowych `random_state=3` (patrz strona 34) i wytrenujmy na nich model naiwnego gaussowskiego klasyfikatora Bayesa:

```
model = GaussianNB()
model.fit(X, y);
```

Spróbujmy ocenić jakość otrzymanej klasyfikacji, przez pryzmat pomyłek, jakie otrzymany model mógł popełnić. Mamy cztery możliwości:

a) przypadek w rzeczywistości należy do klasy negatywnej i został przez model poprawnie sklasyfikowany jako należący to tejże klasy, a więc jako **prawdziwie negatywny** (ang. *true negative* – TN),
b) przypadek w rzeczywistości należy do klasy negatywnej, jednak został przez model błędnie zaliczony do klasy pozytywnej, a więc jako **fałszywie pozytywny** (ang. *false positive* – FP),
c) przypadek w rzeczywistości należy do klasy pozytywnej, jednak został przez model błędnie zaliczony do należący klasy negatywnej, a więc jako **fałszywie negatywny** (ang. *false negative* – FN),
d) przypadek w rzeczywistości należy do klasy pozytywnej i został przez model poprawnie sklasyfikowany jako należący to tejże klasy, a więc jako **prawdziwie pozytywny** (ang. *true positive* – TP).

Zliczając, ile przypadków ze zbioru uczącego mieści się w jednej z tych czterech grup, otrzymujemy tzw. macierz pomyłek (ang. *confusion matrix*):

		Klasa przewidziana	
		negatywna	*pozytywna*
Klasa rzeczywista	*negatywna*	TN	FP
	pozytywna	FN	TP

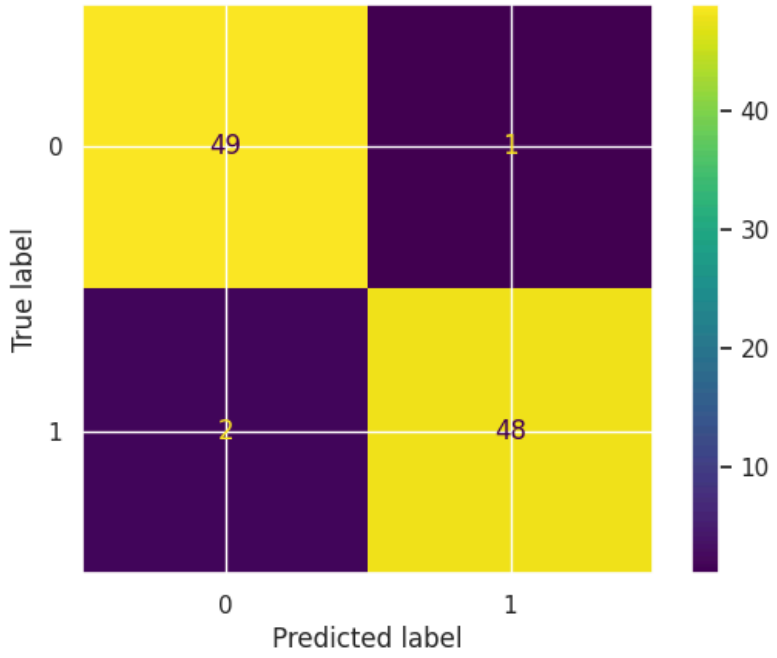
Wartości w niej umieszczone mogą być dane jako bezwzględna liczba przypadków, bądź też jako ich udział (procentowy) w całości zbioru. Przekątna macierzy przedstawia zatem wszystkie przypadki, które zostały prawidłowo przewidziane. Zobaczmy, jak prezentuje się ona dla naszego klasyfikatora – musimy najpierw jednak znaleźć wartości etykiet, przewidziane przez model:

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, \
  recall_score, f1_score, ConfusionMatrixDisplay, RocCurveDisplay
y_pred = model.predict(X)
confusion_matrix(y, y_pred)
```

```
array([[49,  1],
       [ 2, 48]]
```

Oprócz funkcji, wyznaczającej macierz pomyłek, z pakietu *sklearn.metrics* zaimportowaliśmy szereg innych metryk, które posłużą nam do syntetycznej oceny jakości klasyfikacji. Na razie jednak zobaczmy, jak moglibyśmy macierz pomyłek przedstawić w bardziej atrakcyjnej wizualnie formie:

```
ConfusionMatrixDisplay(confusion_matrix(y, y_pred)).plot();
```



Na bazie macierzy pomyłek możemy wyliczyć następujące wartości syntetyczne metryk[57]:

- **dokładność** (ang. *accuracy* – *ACC*) – udział prawidłowych prognoz (zarówno prawdziwie pozytywnych, jak i prawdziwie negatywnych) wśród całkowitej liczby klasyfikowanych przypadków:
- **precyzja** (ang. *precision, positive predictive value* – *PPV*) – udział liczby prognoz pozytywnych wśród wszystkich przypadków rzeczywiście pozytywnych,
- **czułość** (ang. *recall, sensitivity, true positive rate* – *TPR*) – udział przypadków prawdziwie pozytywnych pośród prognozowanych jako pozytywne,
- **miara** F_1 (ang. F_1*-score*) – średnia harmoniczna pomiędzy precyzją a czułością.

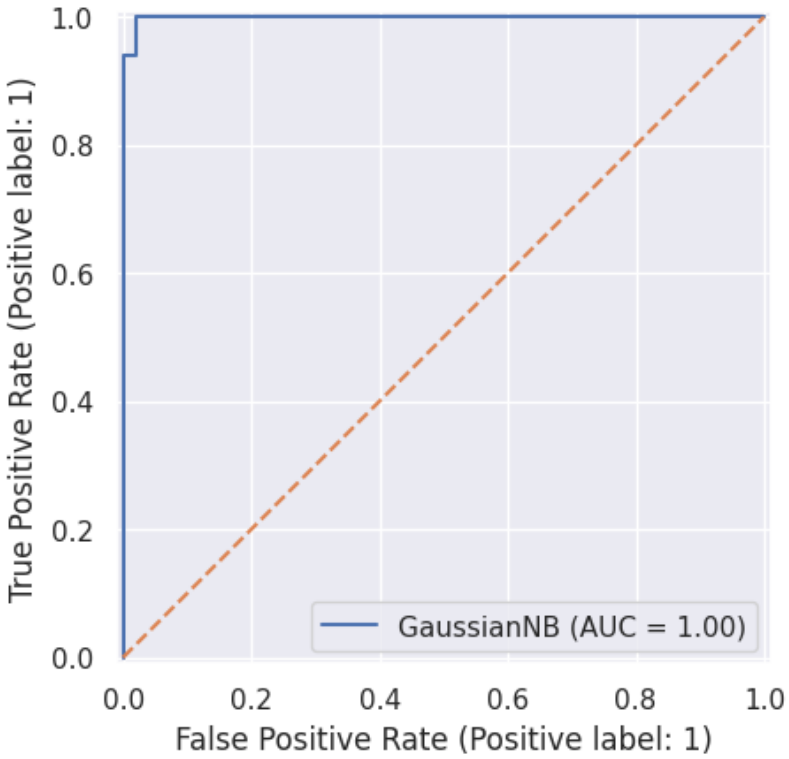
```
print("Dokładność: ", accuracy_score(y, y_pred))
print("Precyzja:   ", precision_score(y, y_pred))
print("Czułość:    ", recall_score(y, y_pred))
print("Miara F1:   ", f1_score(y, y_pred))
```

[57] https://pl.wikipedia.org/wiki/Tablica_pomy%C5%82ek

```
Dokładność:  0.97
Precyzja:    0.9795918367346939
Czułość:     0.96
Miara F1:    0.9696969696969697
```

Otrzymane wartości są zbliżone do siebie, co świadczy o dobrym zrównoważeniu klas w zbiorze uczącym i umacnia nasze zaufanie do modelu. Zdarza się jednak, że tak nie jest – gdyby np. klasa pozytywna oznaczała wystąpienie rzadkiej, ale groźnej choroby w populacji, chcielibyśmy mieć większą pewność niż „50/50", że osoba, którą podejrzewamy o zakażenie nie zostanie omyłkowo zwolniona z izolacji i spowoduje rozprzestrzenianie się patogenów. Zależy nam zatem na zwiększeniu precyzji modelu. Może to jednak spowodować (i zazwyczaj tak się dzieje), że osoby zdrowe będą niepotrzebnie hospitalizowane (niska czułość). To właśnie miara F_1 ma za zadanie zrównoważenie tych dwóch tendencji – w przypadku modeli probabilistycznych (a takim jest m.in. klasyfikator Bayesa) możemy wybrać pewien próg prawdopodobieństwa, powyżej którego przypadek uznajemy jako pozytywny i zrobić to w taki sposób, by zmaksymalizować wartość F_1. Wciąż jednak nasza ocena jest **punktowa** i porównanie w ten sposób różnych modeli może nie być miarodajne – spróbujmy zatem dokonać porównania **globalnego**, tworząc wykres zależności czułości od swoistości (zwanej też specyficznością, ang. *specificity, true negative rate – TNR*), będącej odpowiednikiem czułości dla klasy negatywnej i określającej udział przypadków prawdziwie negatywnych pośród prognozowanych jako negatywne. Wykres ten nosi nazwę **krzywej ROC** – im większe pole powierzchni (zwane AUC – ang. *area under curve*), zawartej między tą krzywą a osiami wykresu, tym lepsza (globalnie) jakość klasyfikatora. Oto, jak ów wykres prezentuje się dla naszego modelu:

```
RocCurveDisplay.from_estimator(model, X, y)
plt.gca().plot([0, 1], [0, 1], '--', c='C1');
```



Pomarańczowa linia przerywana na powyższym rysunku przedstawia wykres krzywej ROC dla klasyfikatora losowego, w wartością AUC = 0.5. Na osi poziomej odłożono wartość *FPR* = 1 – *TNR* (ang. *false positive rate*). Otrzymany model jest prawie idealny – krzywa ROC pokrywa się praktycznie z osiami wykresu i nawet niewielka utrata specyficzności daje prawie 100-procentową czułość.

11. Ćwiczenia

W poprzednich rozdziałach budowaliśmy modele predykcyjne: regresji – w przypadku zmiennej celu o charakterze ciągłym (liczbowym, przyjmującej wartości rzeczywiste) oraz klasyfikacji – dla zmiennej celu o charakterze kategorycznym, (przyjmującej wartości etykiet ze zbioru skończonego), trenowaliśmy je z użyciem dostępnych zbiorów danych a następnie ocenialiśmy wydajność tychże modeli na owych zbiorach treningowych. W praktyce jednak zależy nam na ocenie tego, jak model „poradzi sobie", dokonując predykcji na nowych danych, generalizując zależności pomiędzy zmiennymi wejściowymi X a zmienną celu y, których nauczył się w trakcie treningu. Musimy zatem dysponować zbiorem danych testowych, który najczęściej wydzielamy z całego zbioru danych, którym dysponujemy (jak opisano w Rozdziale 7), bądź – jak to uczynimy teraz – wygenerujemy z analogicznego rozkładu statystycznego, jak dla danych treningowych (w przypadku, gdy są one generowane sztucznie, w celu zaprezentowania możliwości wybranych algorytmów uczenia maszynowego).

Ćwiczenie 1: Ocena modelu regresji

Wylicz metryki RMSE oraz R^2 modelu regresji liniowej (wytrenowanego w Rozdziale 9) dla danych testowych, wczytanych z pliku „reglin_1_test.csv". Zwizualizuj jego zawartość za pomocą wykresu punktowego, nanieś na wykres prostą regresji. Porównaj otrzymane wyniki z tymi, które zostały wyliczone wcześniej dla zbioru treningowego.

Ćwiczenie 2: Trenowanie modelu regresji nieliniowej

Wytrenuj modele regresji liniowej na zbiorach danych, zawartych w plikach „reglin_2.csv" oraz „reglin_3.csv" a następnie wylicz dla nich metryki RMSE oraz R^2. Jak oceniasz jakość otrzymanych modeli? Spróbuj odpowiedzieć na to pytanie, wizualizując wyniki, jak poprzednio. Następnie wygeneruj w sposób sztuczny zmienną $x_2 = x^2$ dla pierwszego z powyższych zbiorów danych oraz zmienne $x_2 = x^2$ i $x_3 = x^3$ dla zbioru drugiego[58]. Ponownie wytrenuj (nowe) modele regresji liniowej i oceń ich wydajność metrykami, jak wyżej – tym razem zrób to również dla testowych zbiorów danych („reglin_2_test.csv" i „reglin_3_test.csv"). Oceń otrzymane wyniki.

Ćwiczenie 3: Trenowanie i ocena modeli klasyfikacji

Wygeneruj za pomocą poniższego kodu zbiór danych, składający się z 500 obserwacji, należących do dwóch klas:

```
from sklearn.datasets import make_blobs
X, y = make_blobs(500, 2, centers=2, cluster_std=1.5, random_state=3)
```

Zaprezentuj otrzymane dane na wykresie punktowym, wyznacz statystyki opisowe zmiennych i zwizualizuj ich rozkłady. Zestandaryzuj dane, podziel na zbiór testowy i treningowy w stosunku 80:20, zachowując proporcje klas w tych zbiorach. Wyucz na zbiorze treningowym znane Ci modele uczenia maszynowego (w tym dobierając różne zestawy hiperparametrów tych modeli) a następnie oceń wcześniej używanymi metrykami (również wizualizując macierze pomyłek i krzywe ROC) ich jakość, porównując wyniki dla zbioru treningowego i testowego.

Ćwiczenie 4: Klasyfikacja chorób serca

Tym razem weźmiemy na warsztat zbiór danych „heart.csv", obejmujący różne wskaźniki zdrowia pacjentów kardiologicznych – m.in. ich wiek, ciśnienie krwi, tętno i inne. Twoim celem jest opracowanie modeli predykcyjnych, które będą w stanie zidentyfikować osoby z chorobą serca. Biorąc pod uwagę poważne implikacje błędnej diagnozy (pominięcia pacjentów, kwalifikujących się do leczenia), głównym celem jest zapewnienie, że dobry model zidentyfikuje wszystkich potencjalnych chorych, czyniąc metrykę czułości klasyfikatora dla klasy pozytywnej kluczowym wskaźnikiem oceny jego skuteczności. Przed wytrenowaniem modeli, jak w ćwiczeniu poprzednim, dokonaj eksploracyjnej analizy zbioru danych: odkrycia wzorców, analizy rozkładu zmiennych i związków między nimi (w tym ze zmienną celu)

[58] Możesz to zrobić „ręcznie" lub za pomocą klasy `PolynomialFeatures` pakietu *sklearn.preprocessing* (https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PolynomialFeatures.html)

wraz z wizualizacją za pomocą wykresów oraz wykonaj odpowiednie kroki wstępnego przetwarzania danych:

a) usunięcie nieistotnych cech,
b) zajęcie się brakującymi wartościami (zwróć uwagę, w jaki sposób są one kodowane),
c) potraktowanie wartości odstających,
d) skalowanie (lub inne przekształcenie, np. *binning*) zmiennych ciągłych,
e) kodowanie zmiennych kategorycznych,
f) podział na dane treningowe i testowe.

Podsumuj otrzymane wyniki i wyciągnij wnioski.